

Instruction-Level Parallelism

Static Parallelism

Instruction-Level Parallelism

ILP

ILP: Pipelining

ILP: Pipelining!

**all processors since 1985 have used
pipelining to improve performance**

ILP: Pipelining?
you already knew!

(classic) laundry example

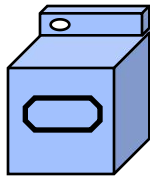
Laundry Example

Ann, Brian, Cathy, Dave

Each has one load of clothes to
wash, dry, fold.



washer
30 mins



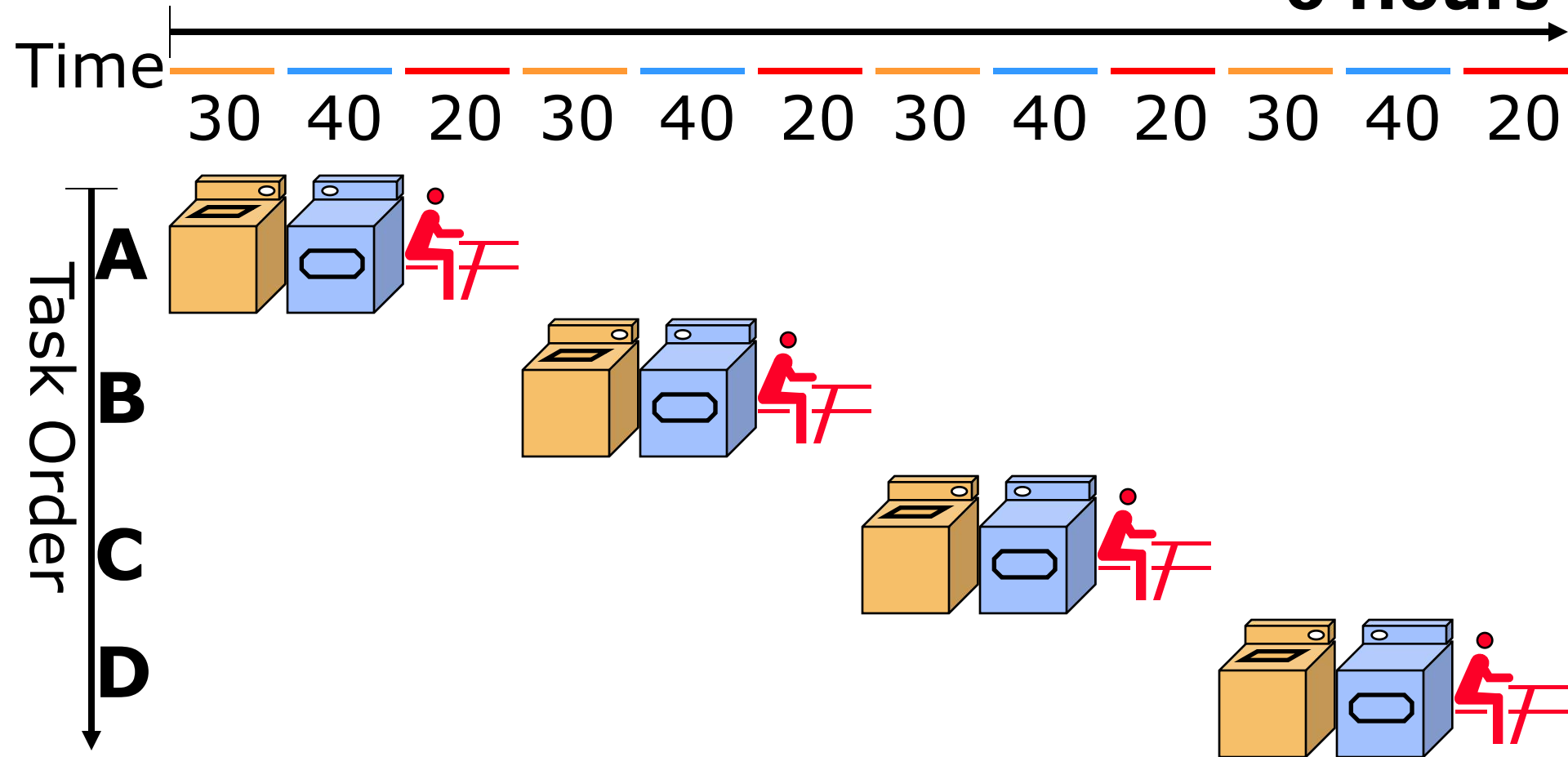
dryer
40 mins



folder
20 mins

Sequential Laundry

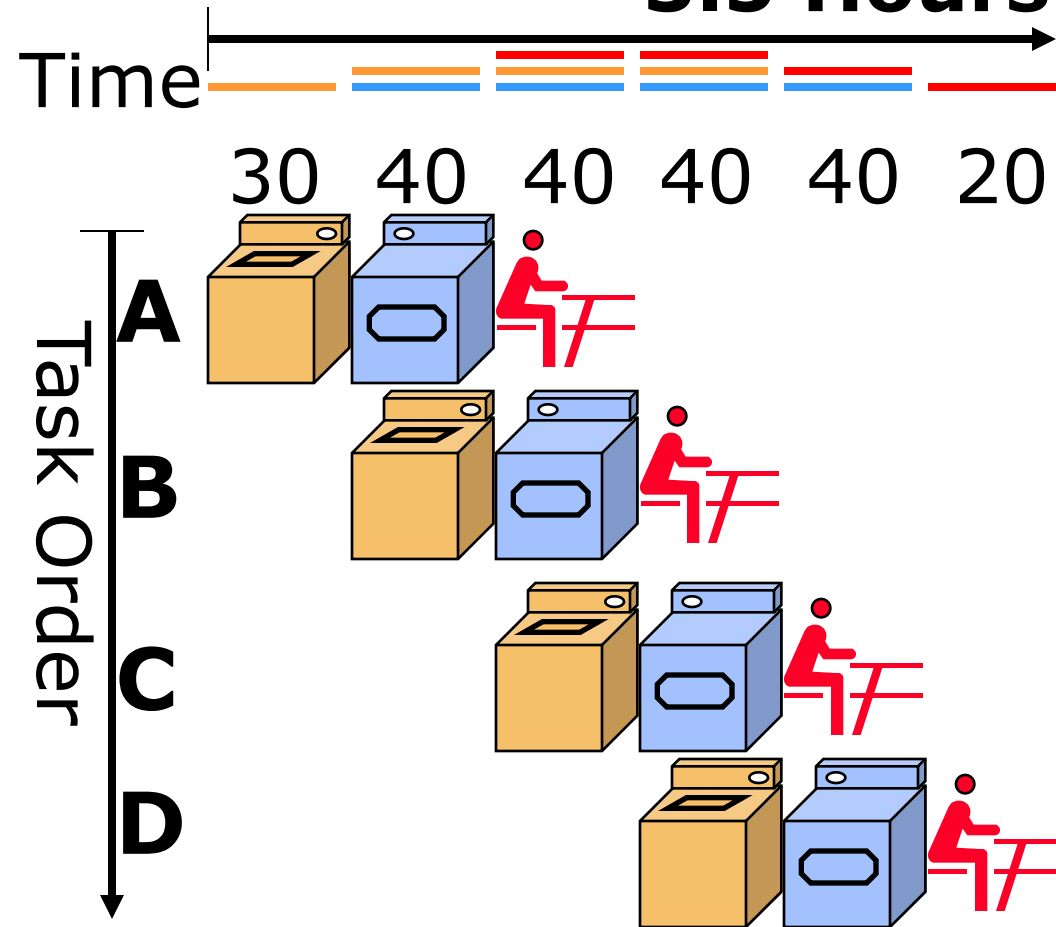
6 Hours



What would you do?

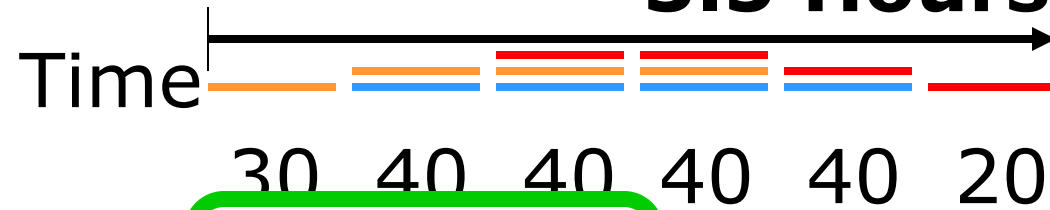
Pipelined Laundry

3.5 Hours

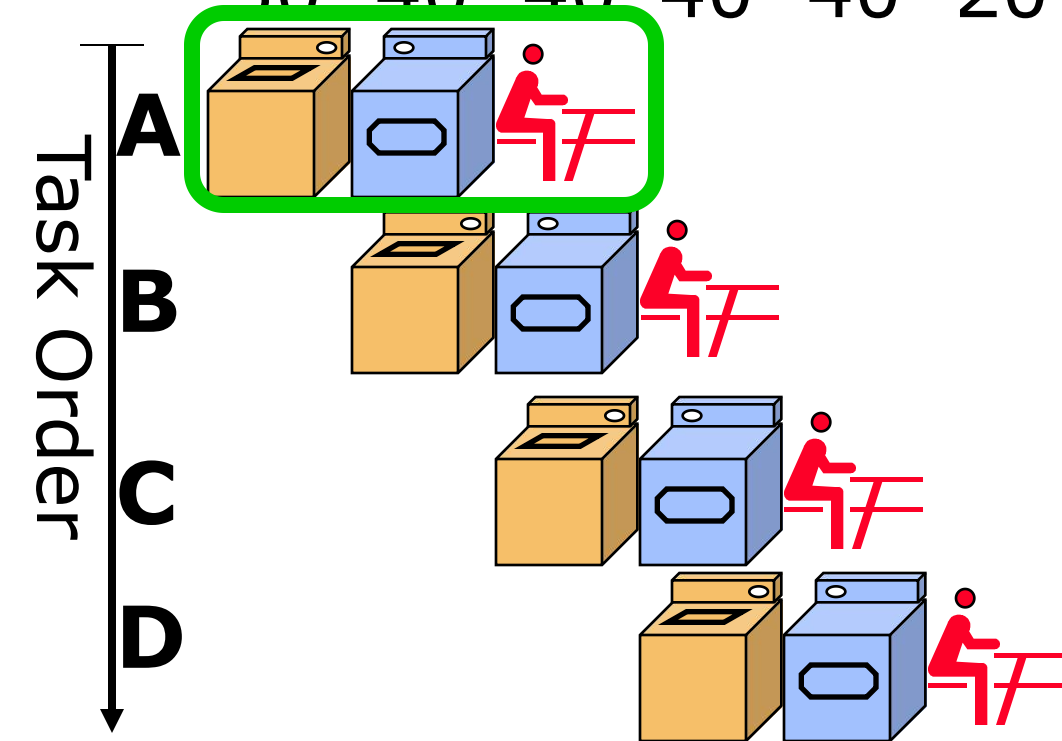


Pipelined Laundry

3.5 Hours Observations

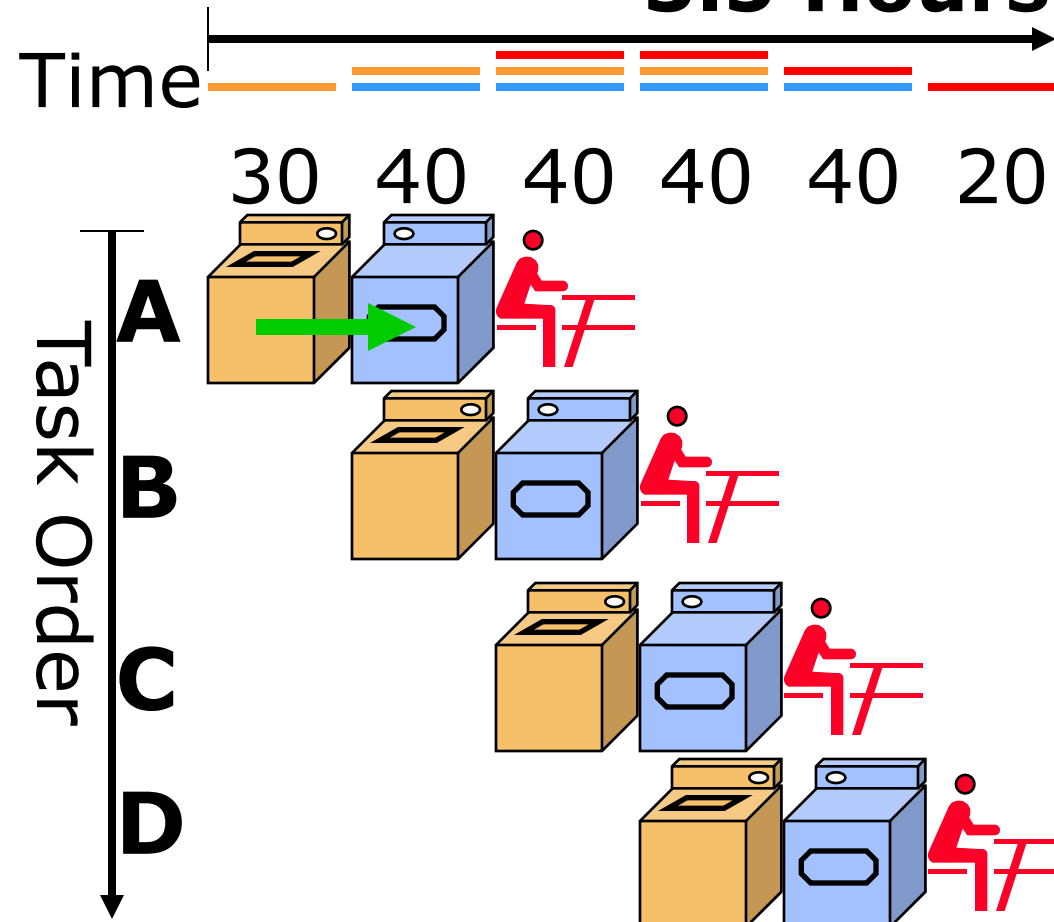


- A task has a series of stages;



Pipelined Laundry

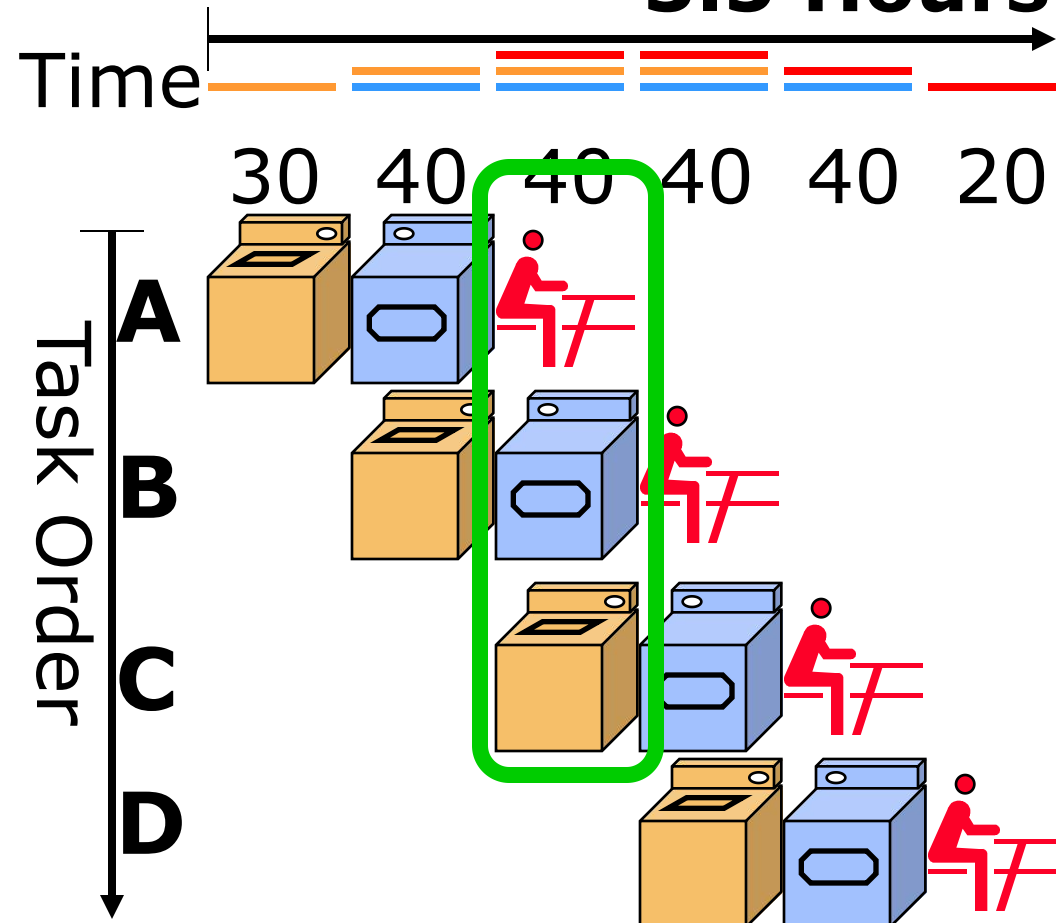
3.5 Hours Observations



- A task has a series of stages;
- Stage dependency: e.g., wash before dry;

Pipelined Laundry

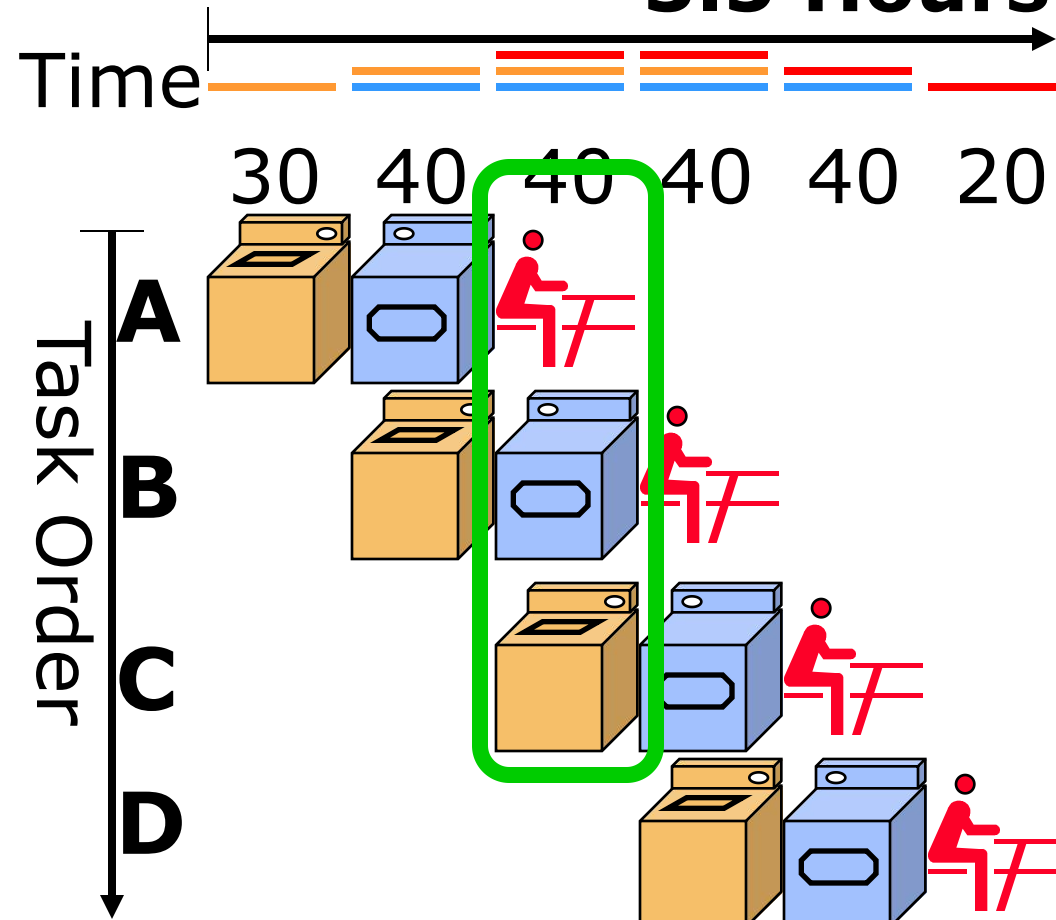
3.5 Hours Observations



- A task has a series of stages;
- Stage dependency: e.g., wash before dry;
- Multi tasks with overlapping stages;

Pipelined Laundry

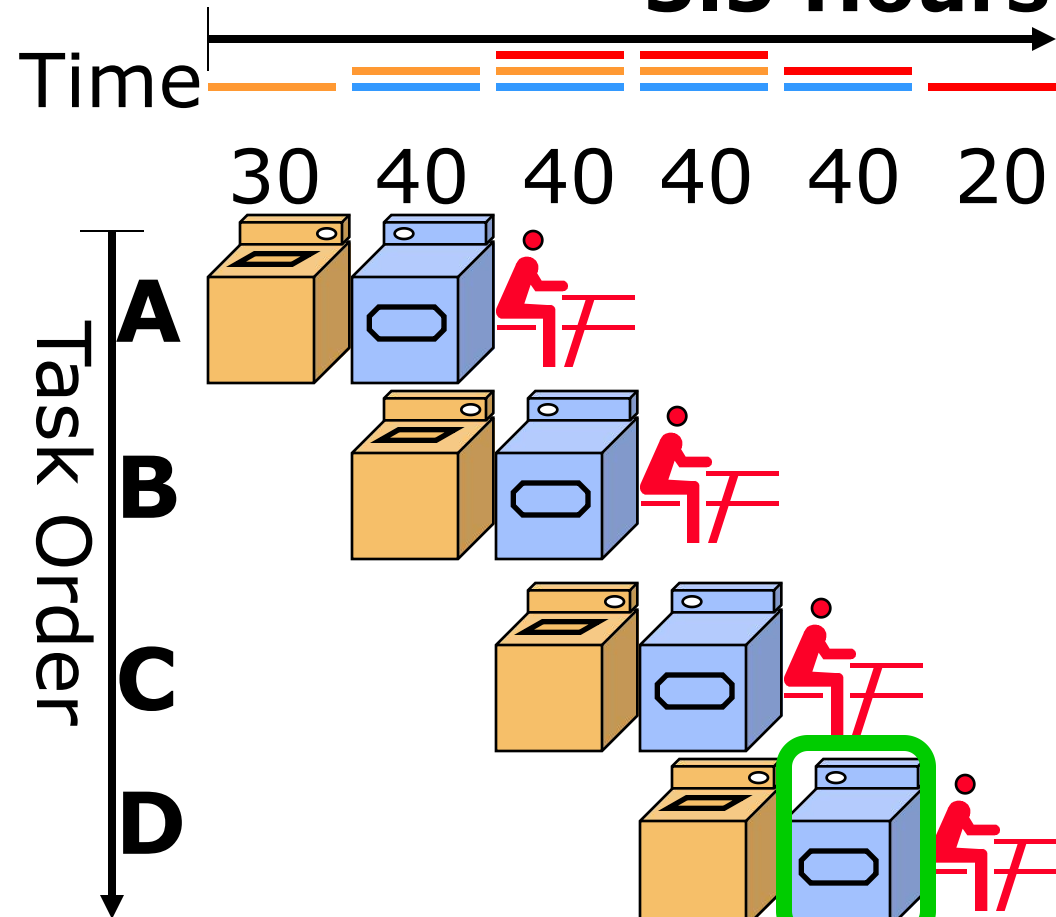
3.5 Hours Observations



- A task has a series of stages;
- Stage dependency: e.g., wash before dry;
- Multi tasks with overlapping stages;
- Simultaneously use diff resources to speed up;

Pipelined Laundry

3.5 Hours Observations



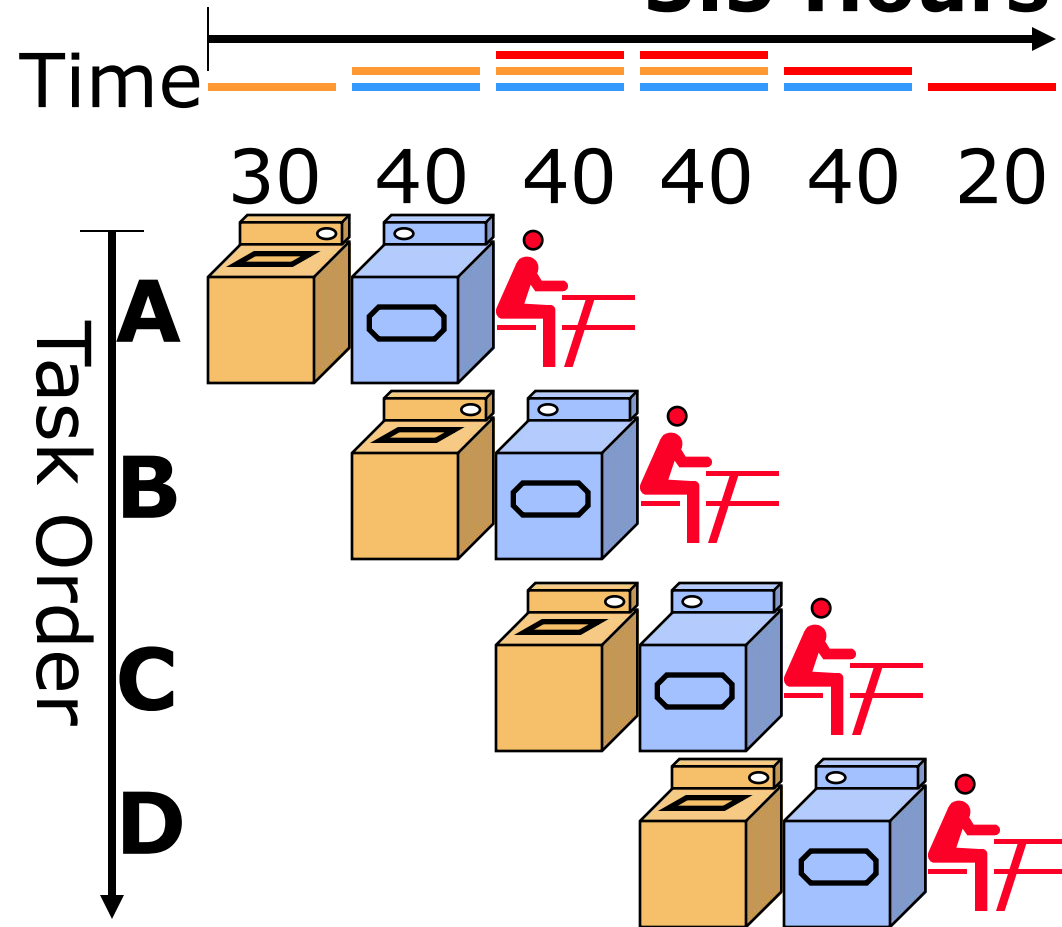
- A task has a series of stages;
- Stage dependency: e.g., wash before dry;
- Multi tasks with overlapping stages;
- Simultaneously use diff resources to speed up;
- Slowest stage determines the finish time;

Pipelined Laundry

3.5 Hours

Observations

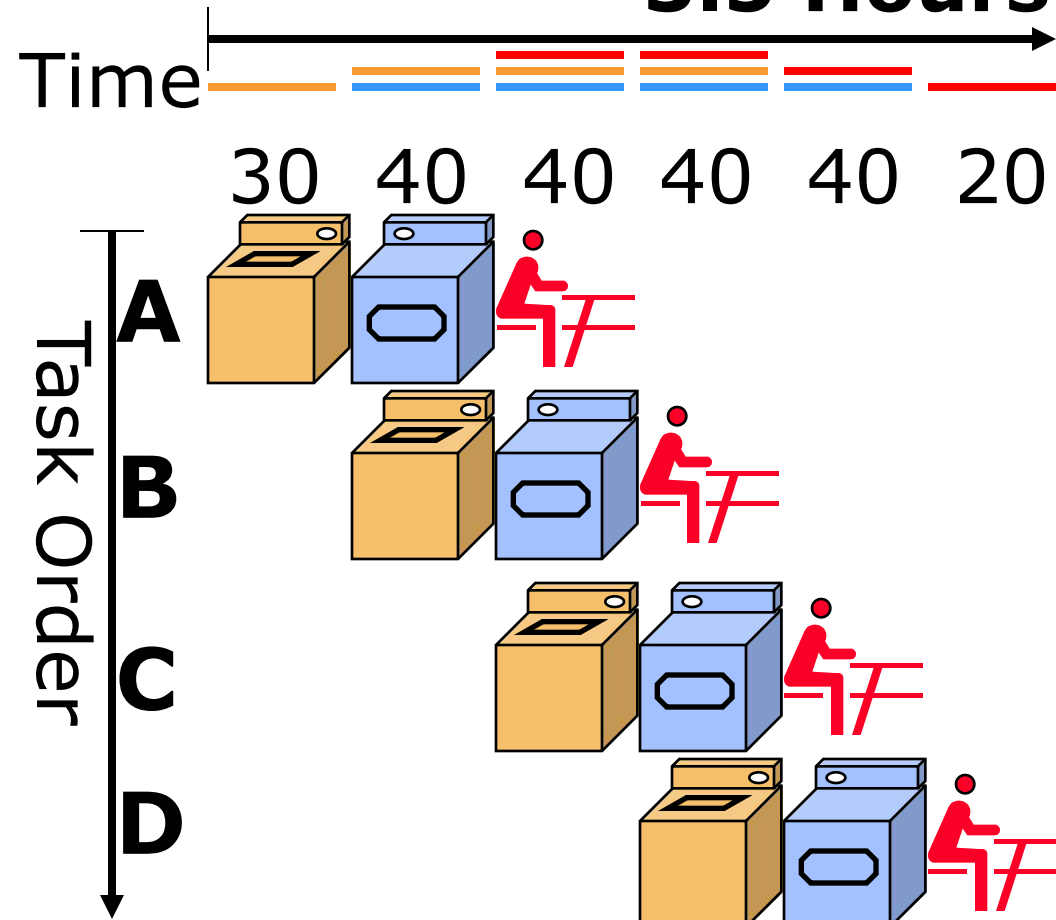
- No speed up for individual task;
e.g., A still takes $30+40+20=90$



Pipelined Laundry

3.5 Hours

Observations



- No speedup for individual task; e.g., A still takes $30+40+20=90$
- But speedup for average task execution time; e.g., $3.5 \times 60 / 4 = 52.5 < 30+40+20=90$

Pipeline Elsewhere: Assembly Line



Auto

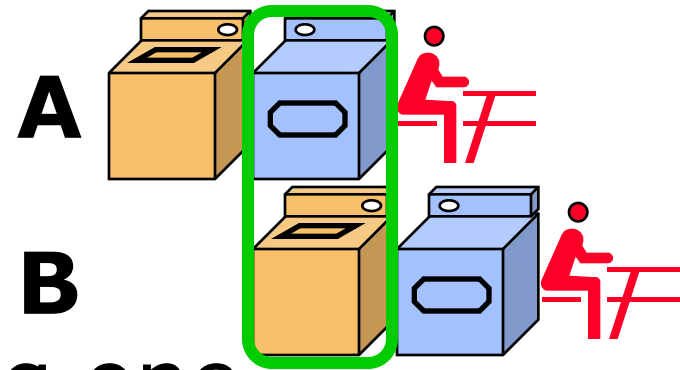


Cola

**What exactly is pipelining
in computer arch?**

Pipelining

- An implementation technique whereby multiple instructions are overlapped in execution.
e.g., B wash while A dry



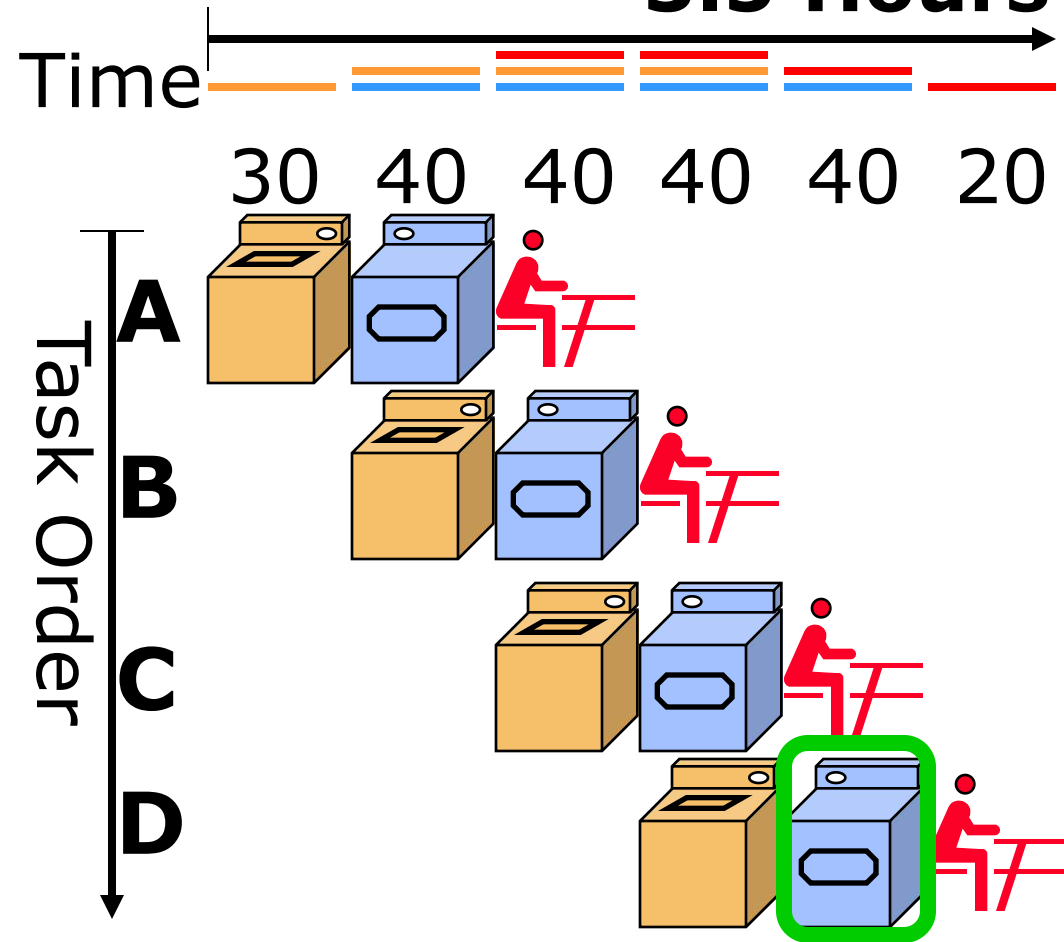
- **Essence:** Start executing one instruction before completing the previous one.
- **Significance:** Make fast CPUs.

Emm, (in)Equality

3.5 Hours

Observations

- A task has a series of stages;
- Stage dependency: e.g., wash before dry;
- Multi tasks with overlapping stages;
- Simultaneously use diff resources to speed up;
- **Slowest stage determines the finish time;**



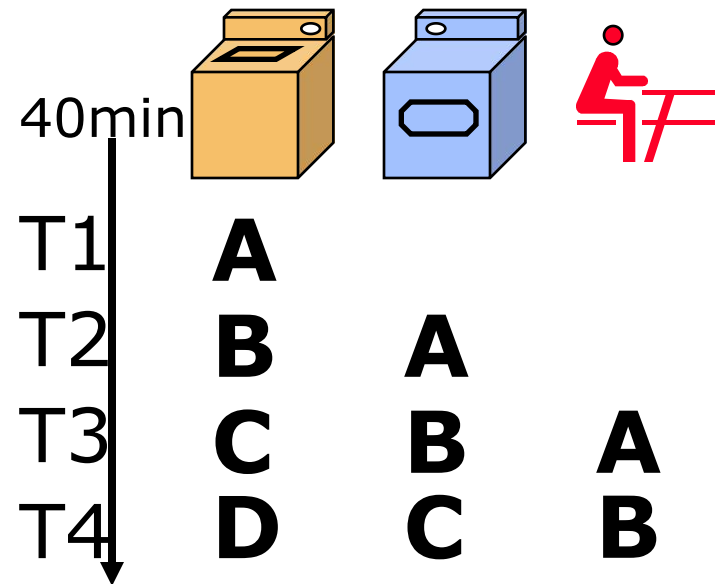
(ideal) Balanced Pipeline

- Equal-length pipe stages/segments

e.g., Wash, dry, fold = 40 mins

per unpipelined laundry time = 40×3 mins

3 pipe stages – wash, dry, fold



Balanced Pipeline

- Equal-length pipe stages

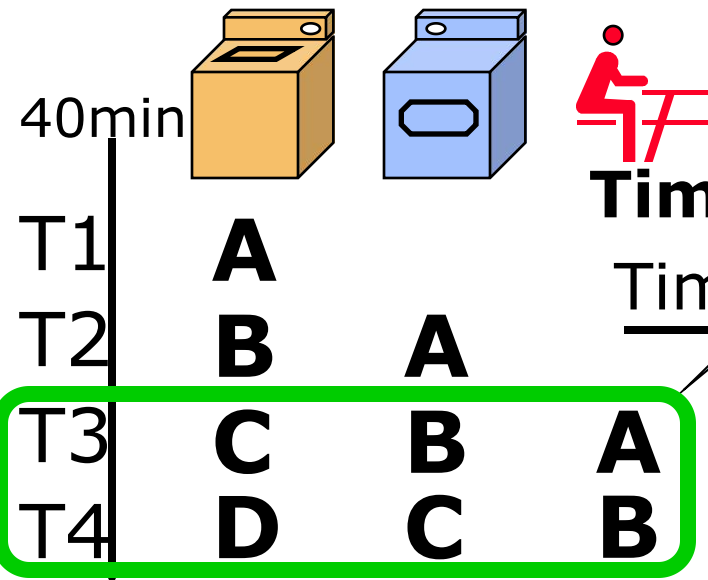
e.g., Wash, dry, fold = 40 mins

per unpipelined laundry time = 40×3 mins

3 pipe stages – wash, dry, fold

One task/instruction
per 40 mins

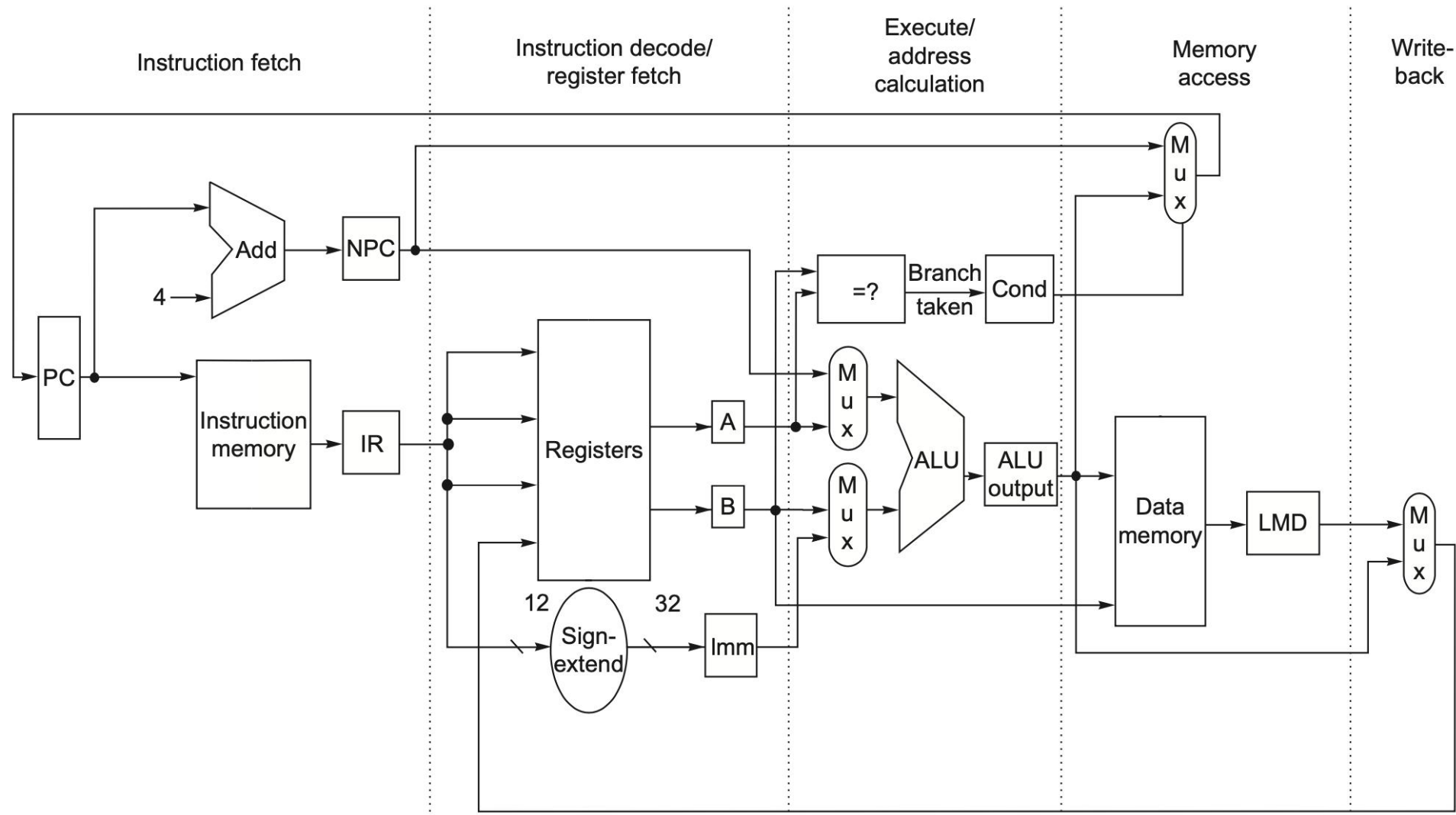
- Performance



$$\text{Time per instruction by pipeline} = \frac{\text{Time per instr on unpipelined machine}}{\text{Number of pipe stages}}$$

$$\text{Speed up by pipeline} = \frac{\text{Time per instr on unpipelined machine}}{\text{Number of pipe stages}}$$

Example RISC-V Data Path



一、上图：Example RISC-V Data Path（单周期数据通路）

1. 核心结构：5 级指令执行阶段（无流水线）

单周期设计中，一条指令在一个时钟周期内完成全部 5 个阶段，所有硬件资源为单条指令独占：

二、下图：Pipelined RISC-V Data Path（流水线数据通路）

1. 核心改造：插入流水线寄存器，实现指令重叠执行

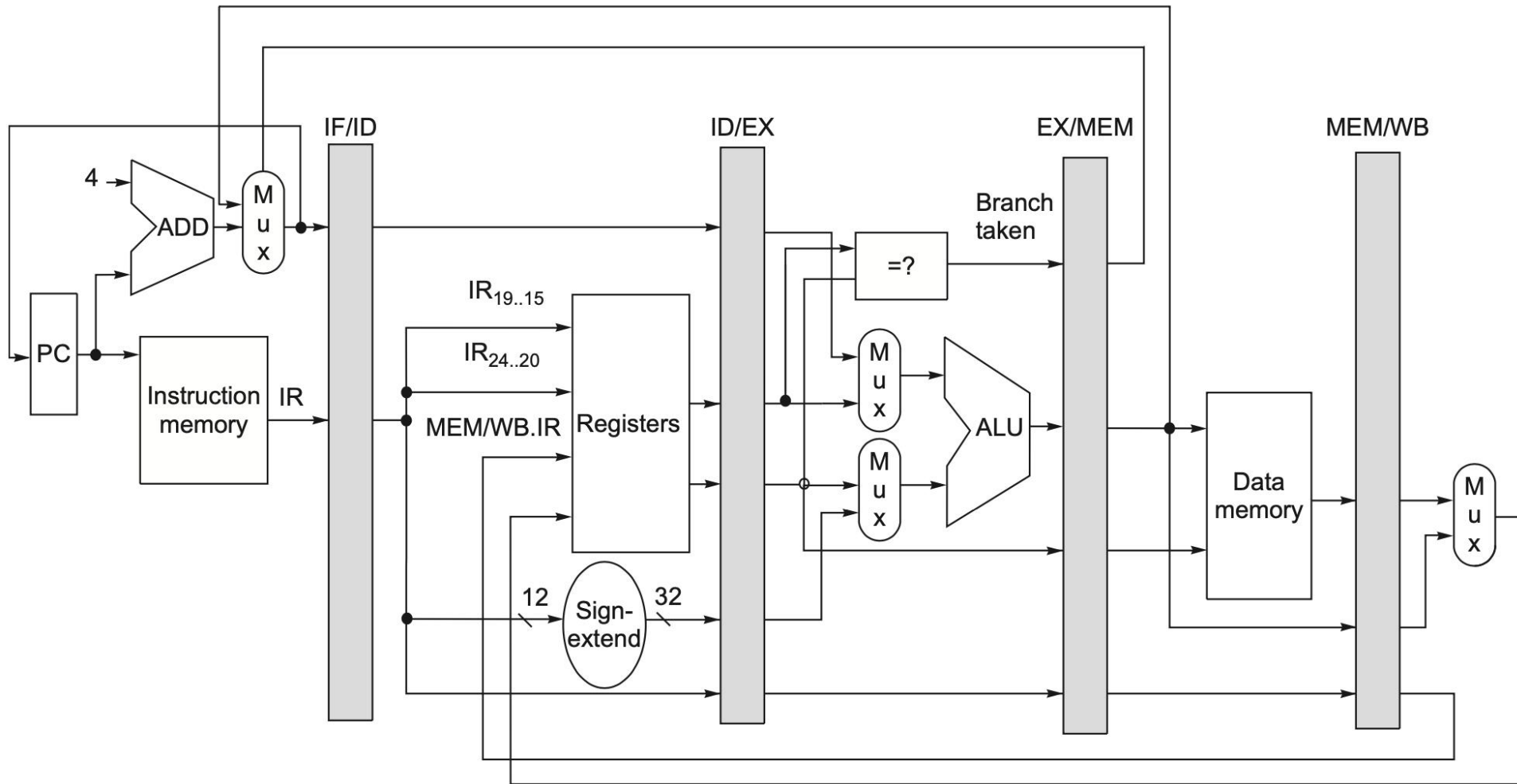
流水线设计将 5 个阶段拆分为独立的流水段，每个时钟周期启动一条新指令，多条指令在不同阶段并行执行：

- **关键改造**：在 5 个阶段之间插入 **4 组流水线寄存器（灰色竖条）**：
 - **IF/ID**：锁存取指阶段结果，传递给译码阶段
 - **ID/EX**：锁存译码阶段结果，传递给执行阶段
 - **EX/MEM**：锁存执行阶段结果，传递给访存阶段
 - **MEM/WB**：锁存访存阶段结果，传递给写回阶段
- **核心原理**：**时空重叠**，用硬件资源的时间复用提升吞吐量，时钟周期仅由单个流水段的延迟决定，频率大幅提升

2. 流水线设计的核心特点

- **吞吐量提升**：理想情况下，CPI（每条指令时钟周期数）从单周期的 1 变为流水线的 1（但实际因冲突、冒险会略高），指令吞吐率提升 5 倍
- **硬件资源共享**：多个指令同时使用不同阶段的硬件，效率大幅提升
- **引入新问题**：数据冒险、控制冒险、结构冒险，需要添加旁路（Forwarding）、气泡（Bubble）、分支预测等机制解决

Pipelined RISC-V Data Path



Pipelined RISC-V Data Path

Stage	Any instruction		
IF	$IF/ID.IR \leftarrow Mem[PC]$ $IF/ID.NPC, PC \leftarrow (if ((EX/MEM.opcode == branch) \& EX/MEM.cond) \{EX/MEM.ALUOutput\} else \{PC+4\});$		
ID	$ID/EX.A \leftarrow Regs[IF/ID.IR[rs1]]$; $ID/EX.B \leftarrow Regs[IF/ID.IR[rs2]]$; $ID/EX.NPC \leftarrow IF/ID.NPC$; $ID/EX.IR \leftarrow IF/ID.IR$; $ID/EX.Imm \leftarrow sign-extend(IF/ID.IR[immediate\ field]);$		
	ALU instruction	Load instruction	Branch instruction
EX	$EX/MEM.IR \leftarrow ID/EX.IR$; $EX/MEM.ALUOutput \leftarrow$ $ID/EX.A\ func\ ID/EX.B$; or $EX/MEM.ALUOutput \leftarrow$ $ID/EX.A\ op\ ID/EX.Imm$;	$EX/MEM.IR\ to\ ID/EX.IR$ $EX/MEM.ALUOutput \leftarrow$ $ID/EX.A + ID/EX.Imm$; $EX/MEM.B \leftarrow ID/EX.B$;	$EX/MEM.ALUOutput \leftarrow$ $ID/EX.NPC +$ $(ID/EX.Imm << 2)$; $EX/MEM.cond \leftarrow$ $(ID/EX.A == ID/EX.B)$;
MEM	$MEM/WB.IR \leftarrow EX/MEM.IR$; $MEM/WB.ALUOutput \leftarrow$ $EX/MEM.ALUOutput$;	$MEM/WB.IR \leftarrow EX/MEM.IR$; $MEM/WB.LMD \leftarrow$ $Mem[EX/MEM.ALUOutput]$; or $Mem[EX/MEM.ALUOutput] \leftarrow$ $EX/MEM.B$;	
WB	$Regs[MEM/WB.IR[rd]] \leftarrow$ $MEM/WB.ALUOutput$;	For load only: $Regs[MEM/WB.IR[rd]] \leftarrow$ $MEM/WB.LMD$;	

在流水线处理器的寄存器表示法里，斜杠 / 就是「流水线级之间的分界符」，读作「到」，用来表示 **「前一个流水段的输出寄存器，同时也是后一个流水段的输入寄存器」**。

举个最直观的例子：

- **IF/ID**：读作 **IF 到 ID**，全称是 **Instruction Fetch / Instruction Decode**
 - 它是**取指阶段 (IF)**的**输出寄存器**，存着 IF 阶段产生的指令、NPC 等结果
 - 同时也是**译码阶段 (ID)**的**输入寄存器**，ID 阶段从这里拿数据干活
- 同理：
 - **ID/EX**：ID → EX（译码到执行）
 - **EX/MEM**：EX → MEM（执行到访存）
 - **MEM/WB**：MEM → WB（访存到写回）

1. IF/ID 寄存器（取指 → 译码）

- 包含的字段（你图里的写法）：
 - **IF/ID.IR**：Instruction Register，存从指令内存取到的完整 32 位 RISC-V 指令
 - **IF/ID.NPC**：Next Program Counter，存计算好的下一条指令地址（PC+4 或分支目标地址）
- 作用：把 IF 阶段的取指结果，完整锁存后送给 ID 阶段，保证 ID 阶段能拿到正确的指令和地址

2. ID/EX 寄存器（译码 → 执行）

- 包含的字段：
 - **ID/EX.A / ID/EX.B**：从寄存器堆读出的两个源寄存器 **rs1 / rs2** 的值
 - **ID/EX.Imm**：符号扩展后的 32 位立即数
 - **ID/EX.NPC / ID/EX.IR**：从 IF/ID 传递过来的 NPC 和完整指令
- 作用：把 ID 阶段的译码、取寄存器结果，锁存后送给 EX 阶段，供 ALU 做运算、计算地址

3. EX/MEM 寄存器（执行 → 访存）

- 包含的字段：
 - **EX/MEM.ALUOutput**：ALU 运算的结果（算术结果 / 内存地址 / 分支目标地址）
 - **EX/MEM.B**：从 ID/EX 传递过来的源寄存器 B 的值（给 Store 指令写内存用）
 - **EX/MEM.cond**：分支指令的条件判断结果（相等 / 不相等）
 - **EX/MEM.IR**：完整指令
- 作用：把 EX 阶段的执行结果，锁存后送给 MEM 阶段，供访存、分支判断使用

4. MEM/WB 寄存器（访存 → 写回）

- 包含的字段：
 - **MEM/WB.ALUOutput**：从 EX/MEM 传递过来的 ALU 结果（给算术指令写回用）
 - **MEM/WB.LMD**：Load Data Register，从数据内存读出的加载数据（给 Load 指令写回用）
 - **MEM/WB.IR**：完整指令
- 作用：把 MEM 阶段的访存结果，锁存后送给 WB 阶段，最终写回寄存器堆

5 级流水线寄存器 位数 / 内容 / 大小 对照表 (RISC-V 32 位)

表格 📄 ⬇️ 🔗			
流水线寄存器	存储内容	总位数	大小特点
IF/ID	指令 (32bit) + PC/NPC (32bit)	64 位	最小
ID/EX	指令 (32) + PC (32) + A (32) + B (32) + Imm (32) + 控制信号	≈160 位	大
EX/MEM	指令 (32) + ALU 结果 (32) + B (32) + cond (1) + 控制信号	≈180 位	最大
MEM/WB	指令 (32) + ALU 结果 (32) + LMD (32) + 控制信号	≈100 位	中等

4 个寄存器 = 整条流水线的 4 个 “暂存站”

所有指令轮流用、按时间用，**共用同一组硬件。**

4. 一句话终极总结

整个 5 级流水线，硬件上只有 4 个流水线寄存器，所有指令共用这 4 个，不是每条指令一组。

极简表格（考试直接写）

表格   		
流水线寄存器	数量	说明
IF/ID	1 个	所有指令共用
ID/EX	1 个	所有指令共用
EX/MEM	1 个	所有指令共用
MEM/WB	1 个	所有指令共用
总计	4 个	全 CPU 就这 4 个

Data Hazard & Forwarding

hazard detection by comparing
the destination and sources of adjacent instructions

Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Data Hazard & Forwarding

hazard detection by comparing
the destination and sources of adjacent instructions

Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Data Hazard & Forwarding

hazard detection by comparing
the destination and sources of adjacent instructions

Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Data Hazard & Forwarding

hazard detection by comparing
the destination and sources of adjacent instructions

Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Data Hazard & Forwarding

hazard detection by comparing
the destination and sources of adjacent instructions

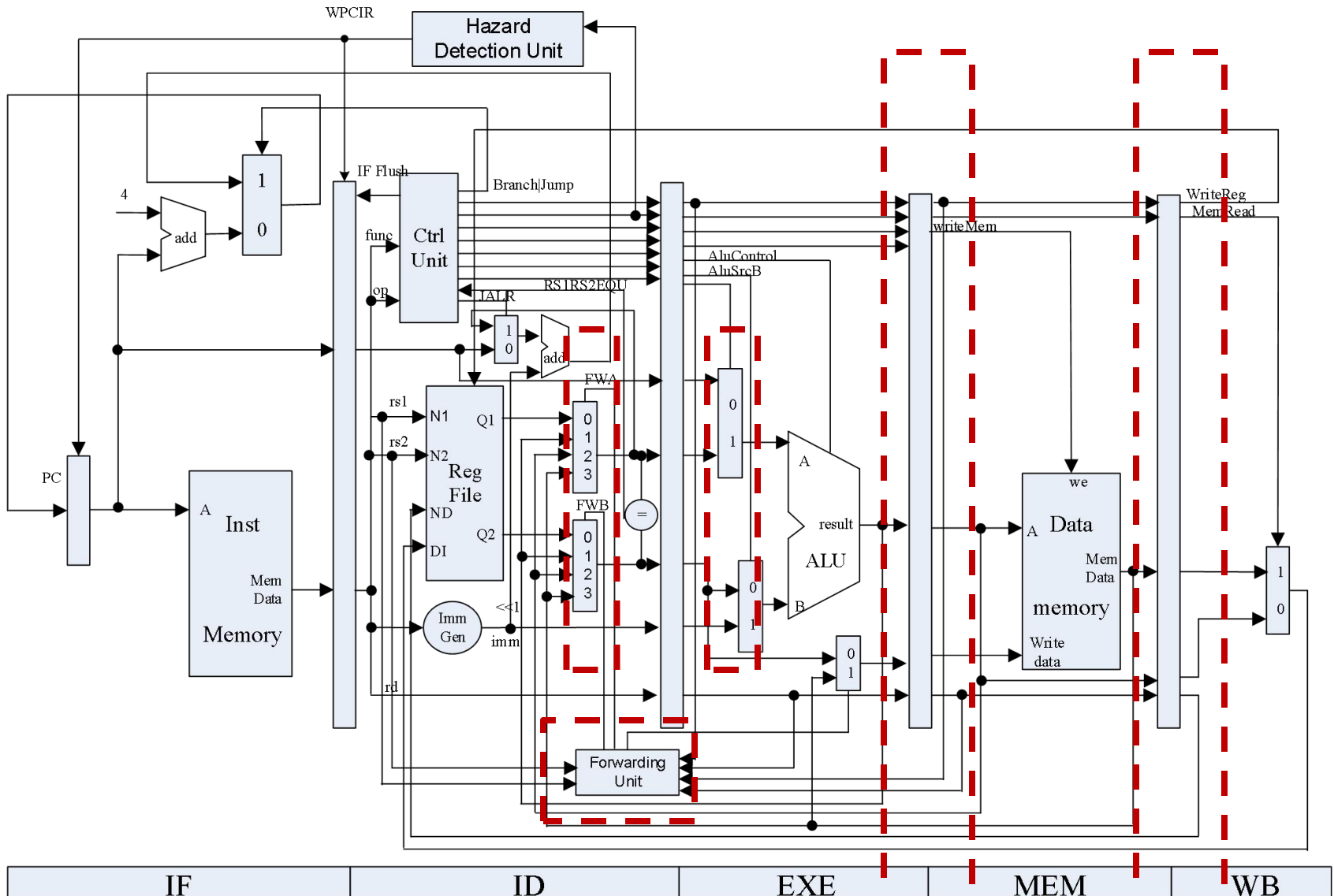
Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Data Hazard & Forwarding

hazard detection by comparing the destination and the sources on the two instructions following the instruction that wrote the destination

Situation	Example code sequence	Action
No dependence	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
Dependence requiring stall	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
Dependence overcome by forwarding	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
Dependence with accesses in order	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

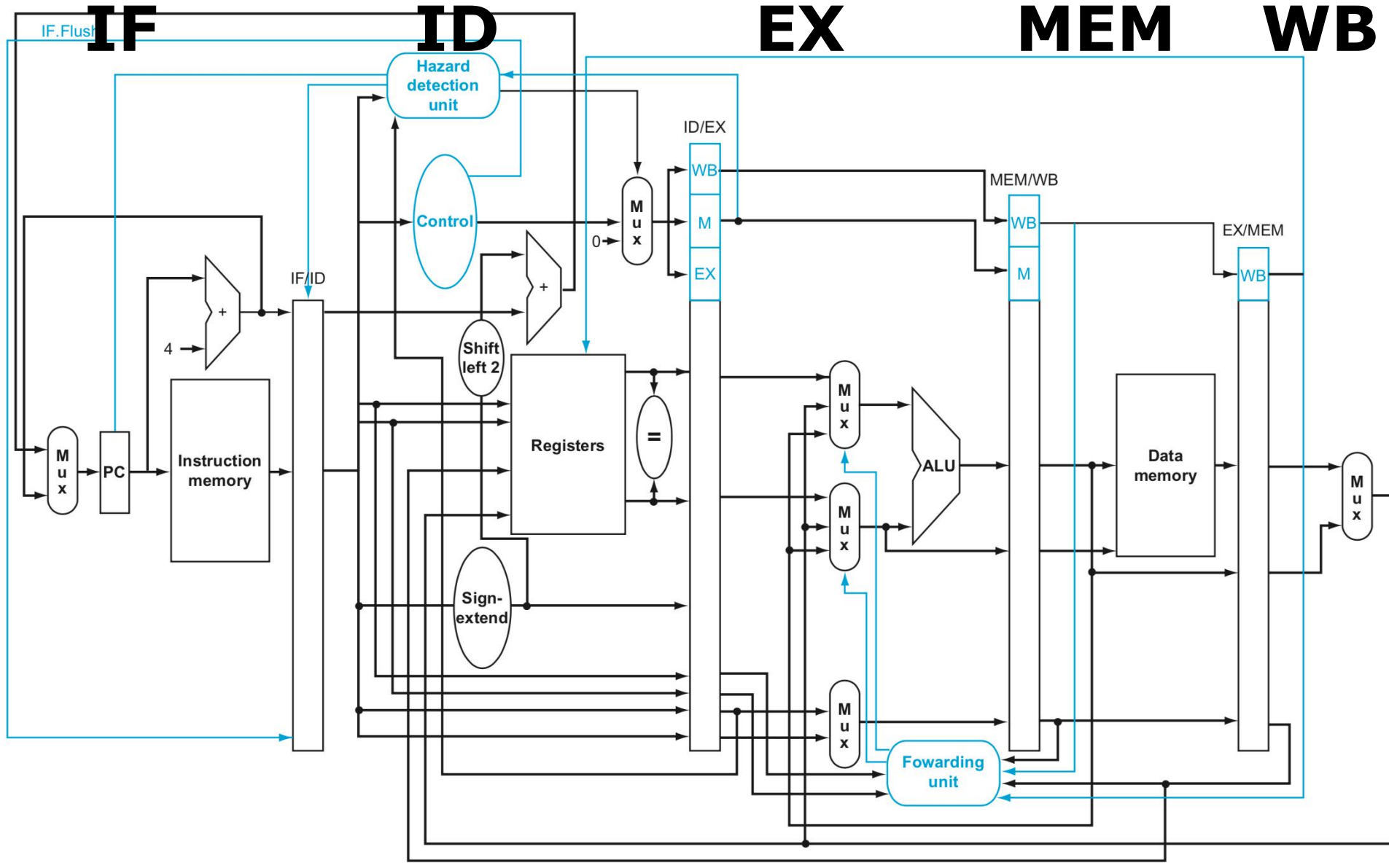
Data Hazard & Forwarding



2. 前递 (Forwarding) 的核心逻辑

- 不等待写回：直接从前序指令的 **EX/MEM**、**MEM/WB** 流水线寄存器中，把刚生成的结果旁路到当前指令的 ALU 输入
- 前递路径 (图中连线):
 1. **EX→EX 前递**：前序指令在 EX 阶段生成的结果，直接旁路到当前指令的 EX 阶段 ALU 输入 (解决 **add** 紧跟 **sub** 的 RAW 冒险)
 2. **MEM→EX 前递**：前序指令在 MEM 阶段的结果，旁路到当前指令的 EX 阶段 (解决隔一条指令的 RAW 冒险)
 3. **WB→EX 前递**：前序指令在 WB 阶段的结果，旁路到当前指令 (解决更远的 RAW 冒险)
- 多路选择器控制：Forwarding Unit 根据指令的 **rs1/rs2** 与后续阶段的 **rd** 匹配情况，控制 MUX 选择前递值或寄存器原值

Integer Op in 1 CC



2. 关键支持机制

这张图集成了之前讨论的所有高级机制，以保证**单周期内正确执行**：

(1) Forwarding Unit（前递单元）

- **位置**：EX 阶段，连接 ID/EX、EX/MEM、MEM/WB 流水线寄存器。
- **作用**：解决**数据冒险（Data Hazard）**。
 - 当后序指令（如 `sub`）需要前序指令（如 `add`）的结果时，Forwarding Unit 直接从 EX/MEM 或 MEM/WB 中读取结果，绕过寄存器堆，送入 ALU 的输入多路选择器。
 - **效果**：无需停顿，确保算术指令在 EX 阶段就能使用最新值，保证 1 CC 完成。

(2) Hazard Detection Unit（冒险检测单元）

- **位置**：ID 阶段。
- **作用**：检测**结构冒险**或**无法通过前递解决的冒险**（如 Load-Use 冒险）。
- **响应**：
 - 检测到冒险时，发送 **IF.Flush** 信号，冲刷 IF 阶段的指令，或插入**气泡（Bubble）**，暂停后续指令，保证数据正确性。
 - 同时处理**分支 / 跳转（Branch/Jump）**，计算目标地址，确保 PC 正确跳转。

(3) 流水线寄存器 (Pipeline Registers)

- 图中清晰标注了 **IF/ID、ID/EX、EX/MEM、MEM/WB** 4 个核心流水线寄存器。
- **作用**：**锁存**每个阶段的输出信号，为下一阶段提供稳定的输入，是实现 1 个时钟周期完成一个阶段的硬件基础。

**What about
floating-point operation?**

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
a slow clock?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
a slow clock?
many logic in FP units?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
a slow clock?
many logic in FP units?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
a slow clock? *slow down integer ops*
many logic in FP units?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
a slow clock? *slow down integer ops*
many logic in FP units?

FP Operation

- **Floating-point (FP) operations** take more time than integer operations do
- To complete an FP op in 1 cc:
 - a slow clock? *slow down integer ops*
 - many logic in FP units? *manufacturing hardness & expensiveness*

二、两大错误方案与致命缺陷（第二点）

PPT 提出了「强行让浮点运算在 1 CC 完成」的两种思路，并标注了其不可行的核心问题（红色字体）：

方案 1：降低时钟频率（a slow clock?）

- 思路：把时钟周期拉得足够长，让浮点运算的长逻辑能在 1 个周期内跑完。
- 致命缺陷（红色标注）：slow down integer ops（整数运算性能被严重拖累）
 - 整数运算占程序执行的绝大多数（>90%），为了少数浮点运算，让所有整数指令的时钟周期翻倍 / 数倍，整体性能暴跌，完全得不偿失。

方案 2：堆砌超大规模浮点逻辑（many logic in FP units?）

- 思路：用超大规模并行逻辑，把浮点运算的多步操作压缩到 1 个周期内完成。
 - 致命缺陷（红色标注）：manufacturing hardness & expensiveness（制造难度极高 + 成本天价）
 - 浮点运算的长逻辑深度无法通过简单并行消除，强行并行会导致浮点单元面积暴增、功耗飙升、良率暴跌，芯片制造成本呈指数级上升，完全不具备工程可行性。
-

Then how?

Multicycle FP Operation

- **FP pipeline**

allow for *a longer latency* for op;
i.e., take >1 cc for EXE;

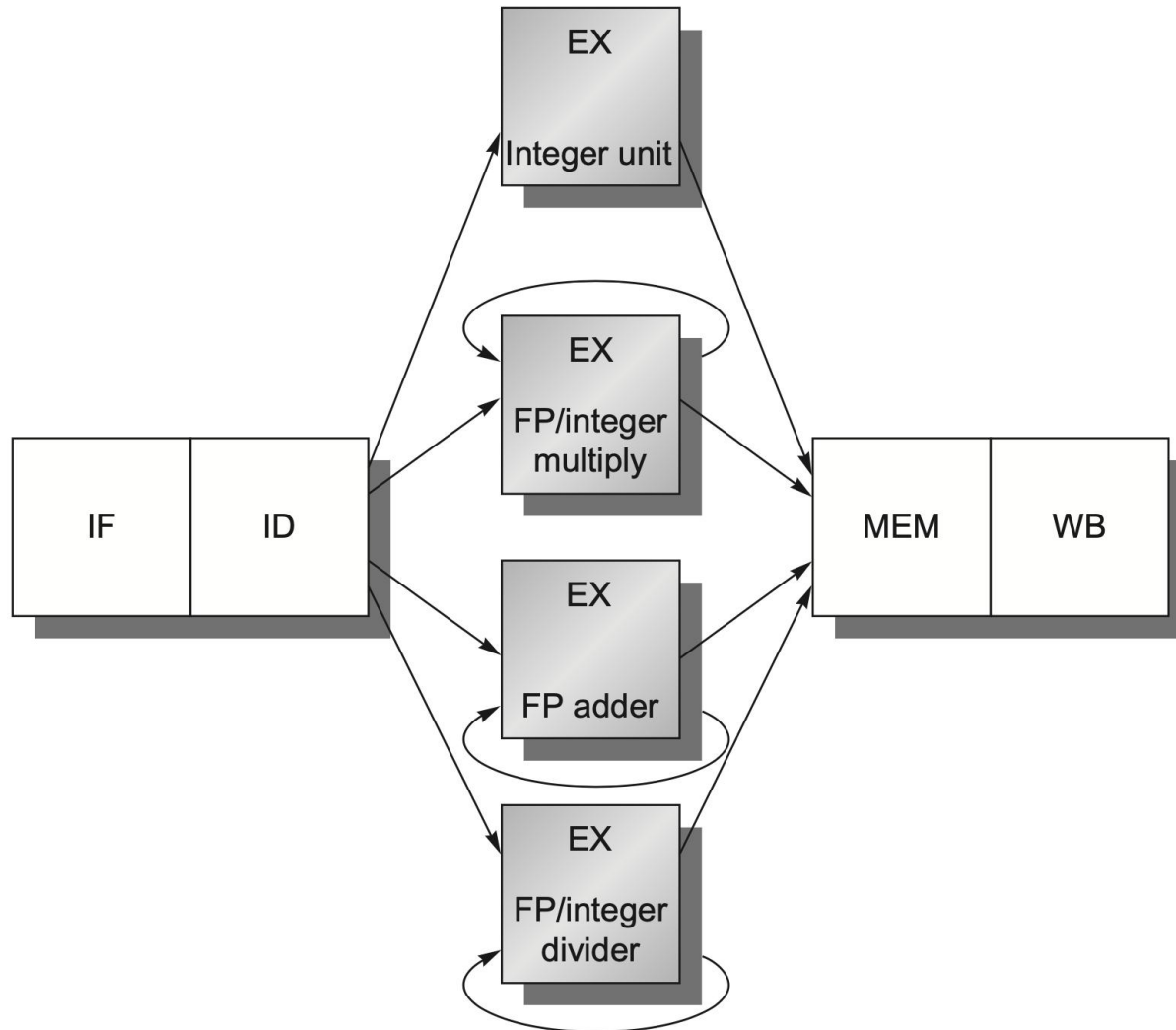
two changes over integer pipeline:

repeat EX;

use multiple FP functional units;

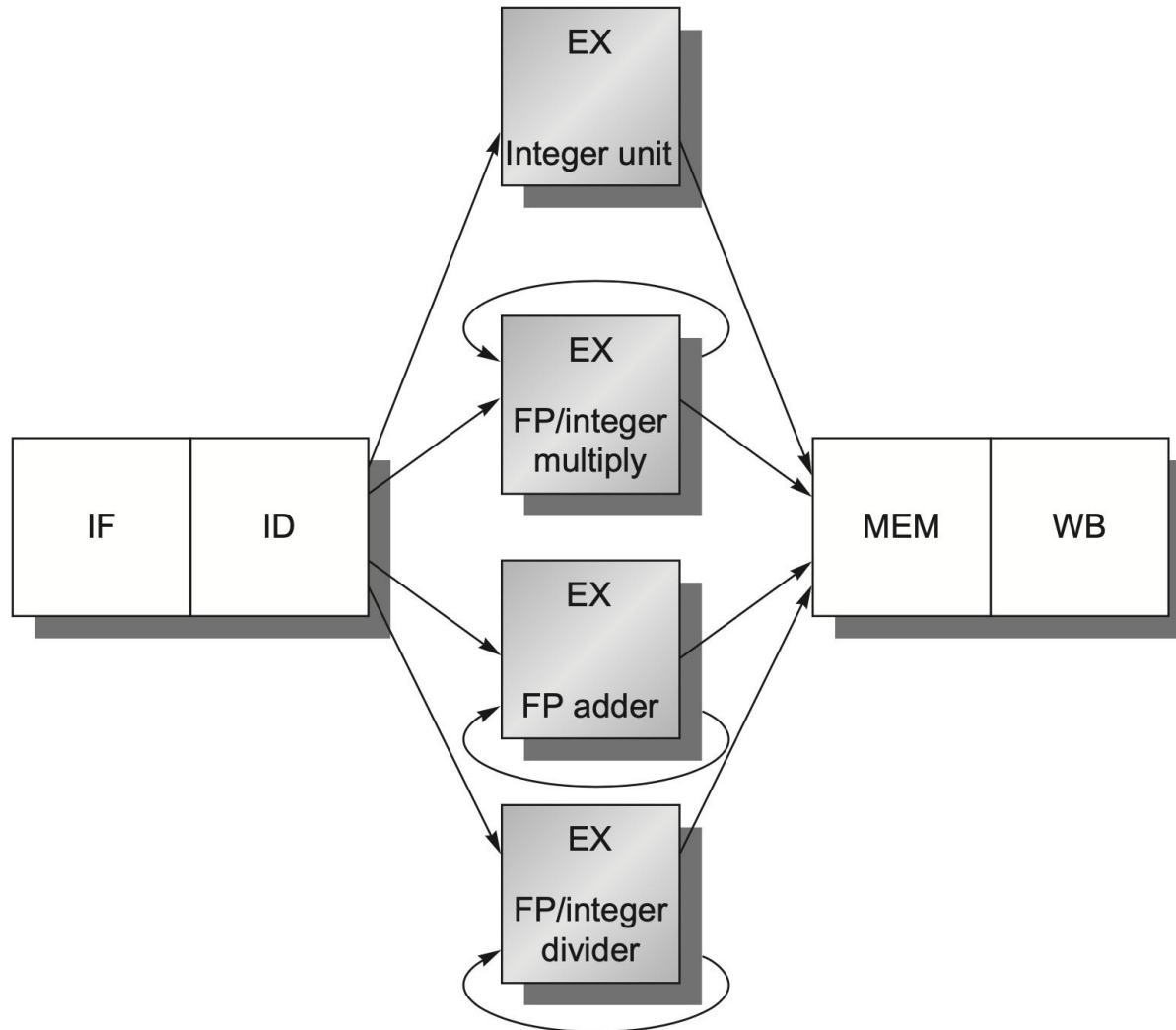
e.g., FP adder, FP divider

FP Pipeline

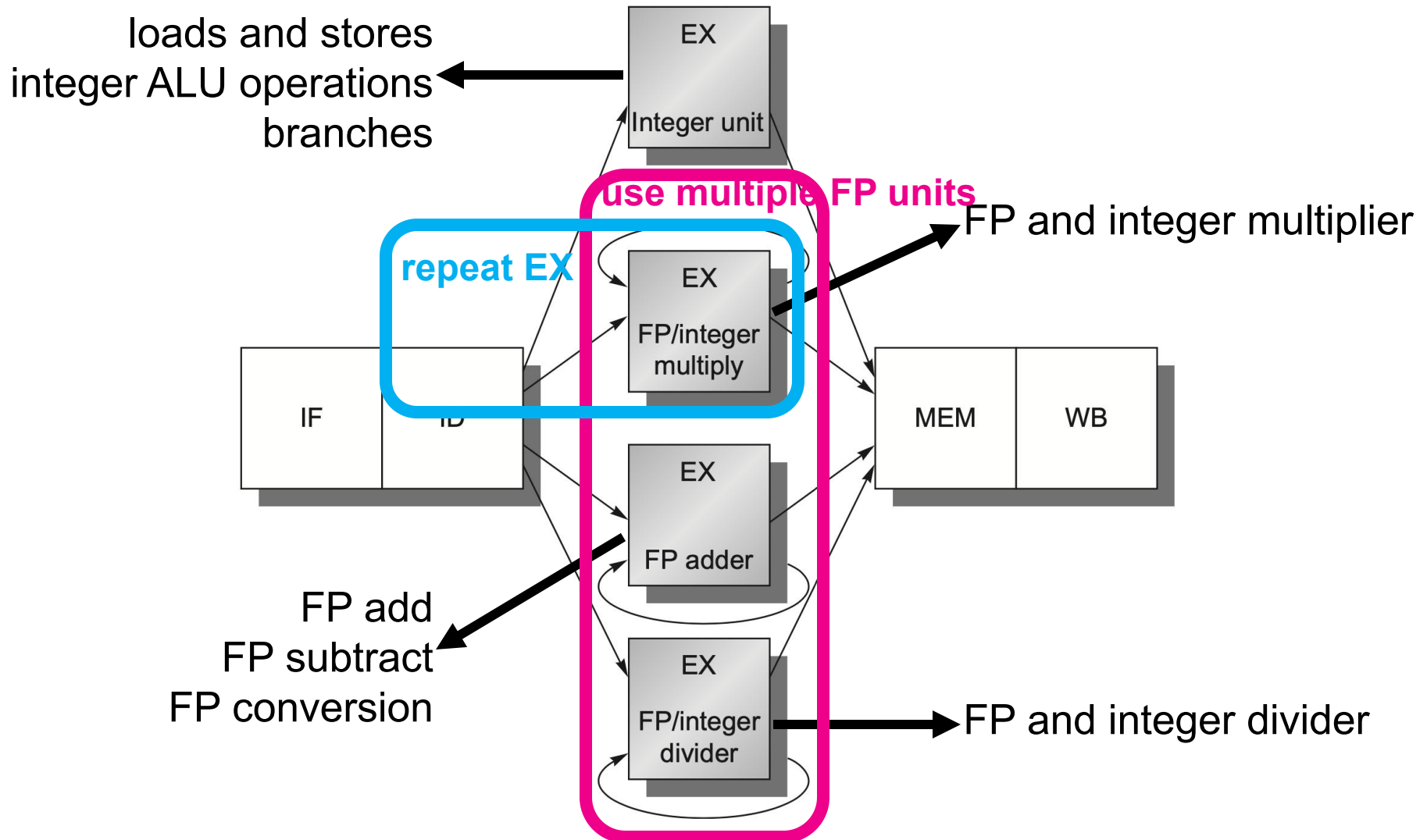


FP Pipeline

how?

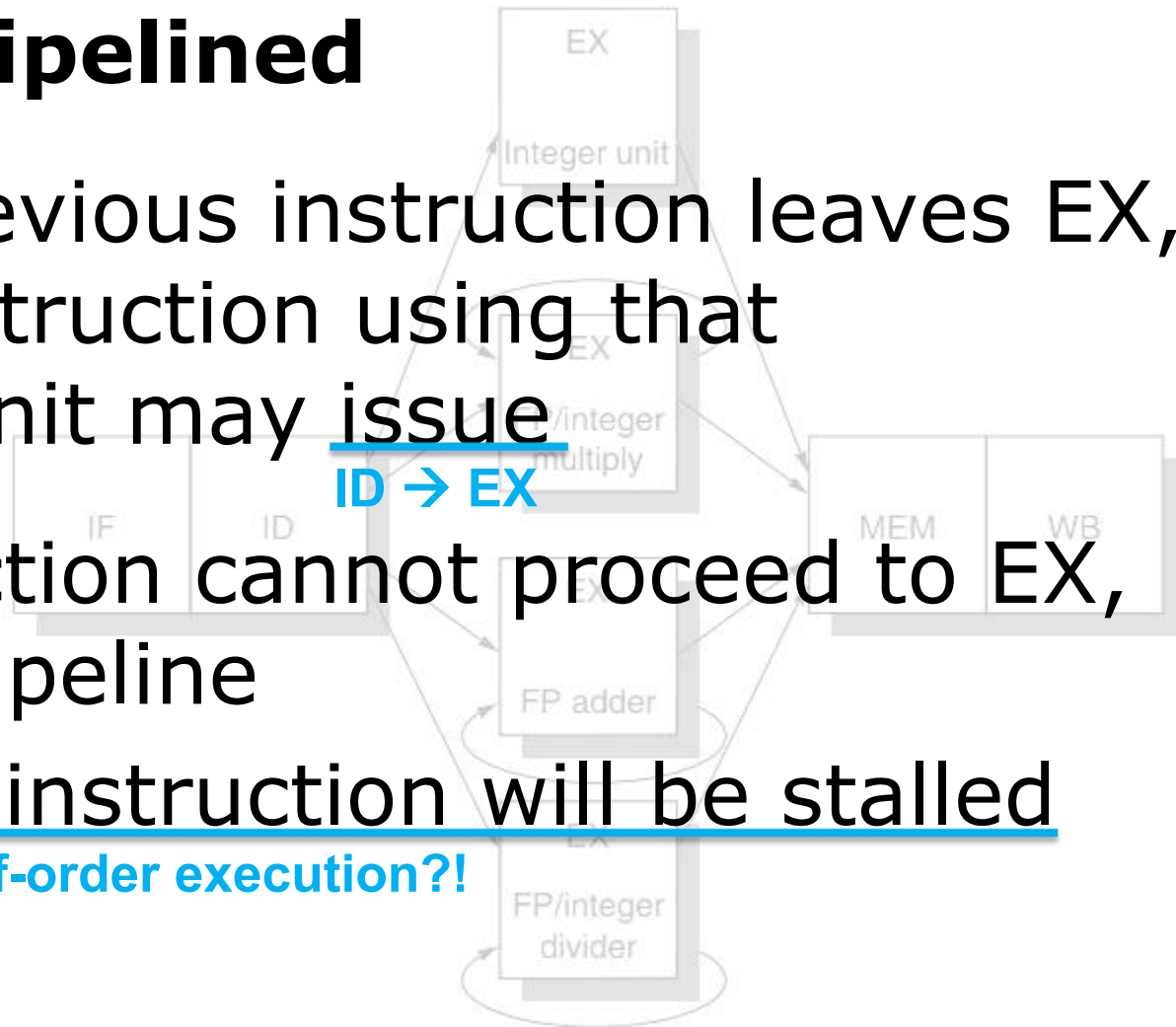


FP Pipeline



FP Pipeline

- **EX is not pipelined**
- Until the previous instruction leaves EX, no other instruction using that functional unit may issue
ID → EX
- If an instruction cannot proceed to EX, the entire pipeline behind that instruction will be stalled
curious about out-of-order execution?!



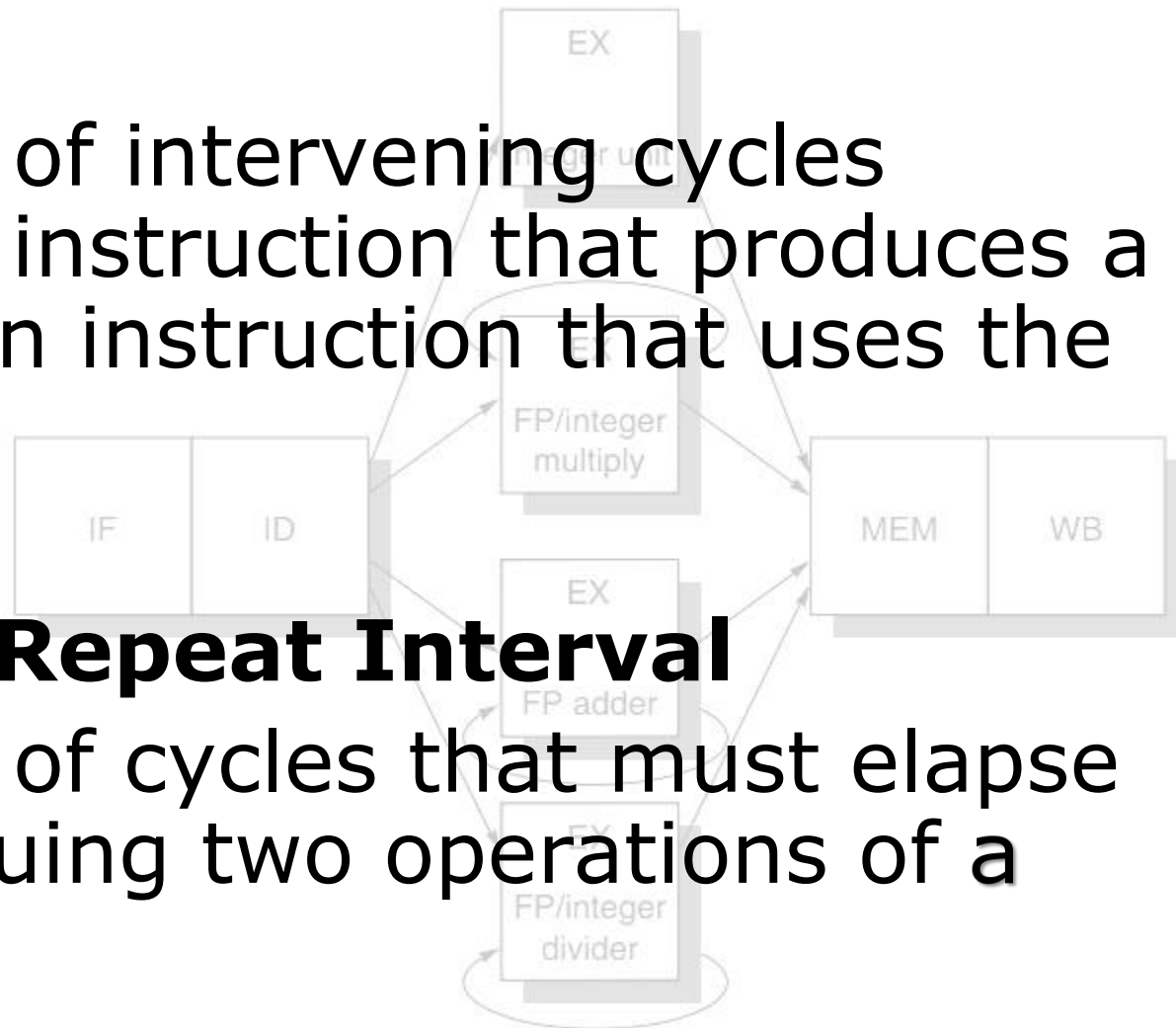
Latency & Ini/Repeat Interval

- **Latency**

the number of intervening cycles between an instruction that produces a result and an instruction that uses the result

- **Initiation/Repeat Interval**

the number of cycles that must elapse between issuing two operations of a given type



Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Essentially, pipeline **latency** is 1 cycle less than the depth of the execution pipeline, which is the number of stages from the **EX stage** to the stage that produces the result

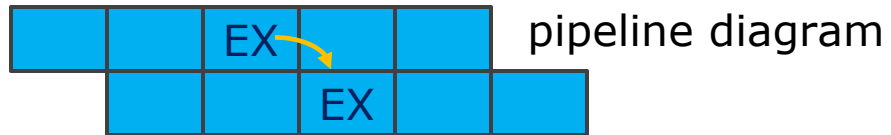
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two (dependent) integer ALU instructions:

ADD R3, R1, R2

ADD R5, R3, R4



Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

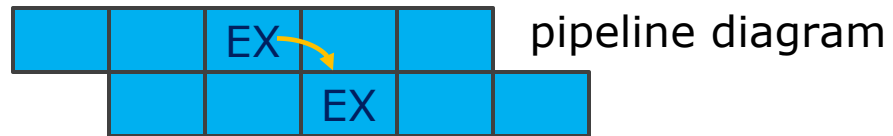
Two (dependent) integer ALU instructions:

ADD R3, R1, R2

ADD R5, R3, R4

Latency: 0

as no intervention to pipeline



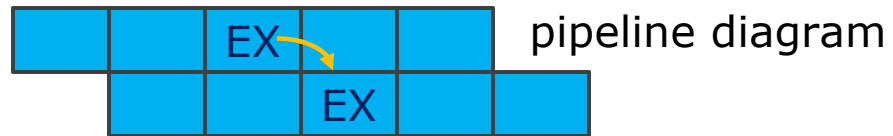
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two (dependent) integer ALU instructions:

ADD R3, R1, R2

ADD R5, R3, R4



Initiation interval: 1

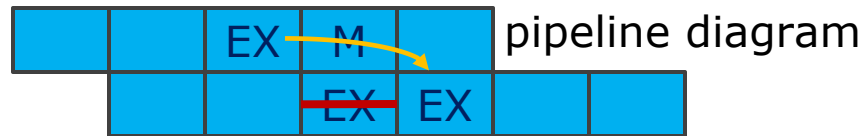
as 2nd ADD has to wait for 1 cc after 1st ADD

Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two (dependent) instructions: Load + ADD

Load R2, 0(R1)
ADD R3, R2, R1



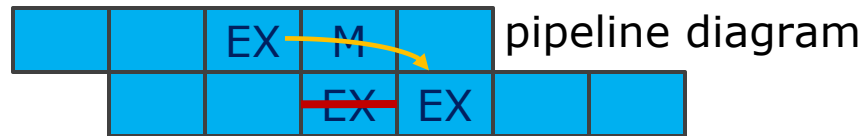
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two (dependent) instructions: Load + ADD

Load R2, 0(R1)

ADD R3, R2, R1



Latency: 1, pipeline is intervened at EX stage as ADD.EX has to wait for 1 cc until Load.MEM

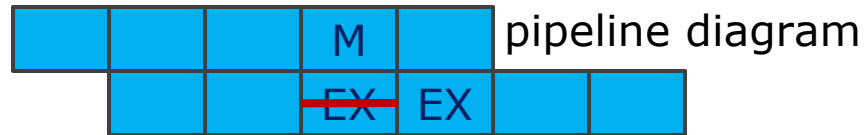
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two (dependent) instructions: Load + ADD

Load R2, 0(R1)

ADD R3, R2, R1



Initiation interval: ?

Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two **same-type** instructions: Load + Load

Load **R2**, 0(R1)

Load **R3**, 0(R1)



Initiation interval: 1

as 2nd Load has to wait for 1 cc after 1st Load

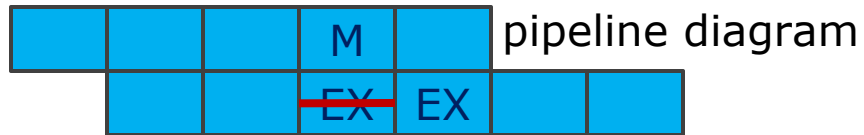
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two **same-type dependent** instructions:

Load **R2**, 0(R1)

Load R3, 0(**R2**)



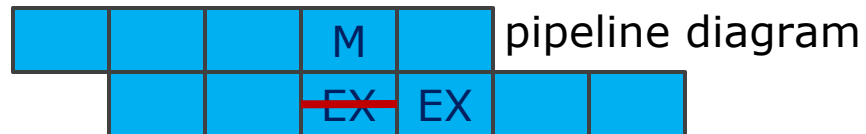
Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Two **same-type dependent** instructions:

Load **R2**, 0(R1)

Load R3, 0(**R2**)



Latency: 1

Initiation interval: still 1 *as pipelined func unit & hazard detection till ID*

Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Essentially, pipeline **latency** is 1 cycle less than the depth of the execution pipeline, which is the number of stages from the **EX stage** to the stage that produces the result

Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Essentially, pipeline **latency** is 1 cycle less than the depth of the execution pipeline, which is the number of stages from the **EX stage** to the stage that produces the result

4 FP ADD
7 FP mul
25 FP div



Latency & Ini/Repeat Interval

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Essentially, pipeline **latency** is 1 cycle less than the depth of the execution pipeline, which is the number of stages from the **EX stage** to the stage that produces the result

4	FP ADD
7	FP mul
<u>24</u>	FP div? 25?

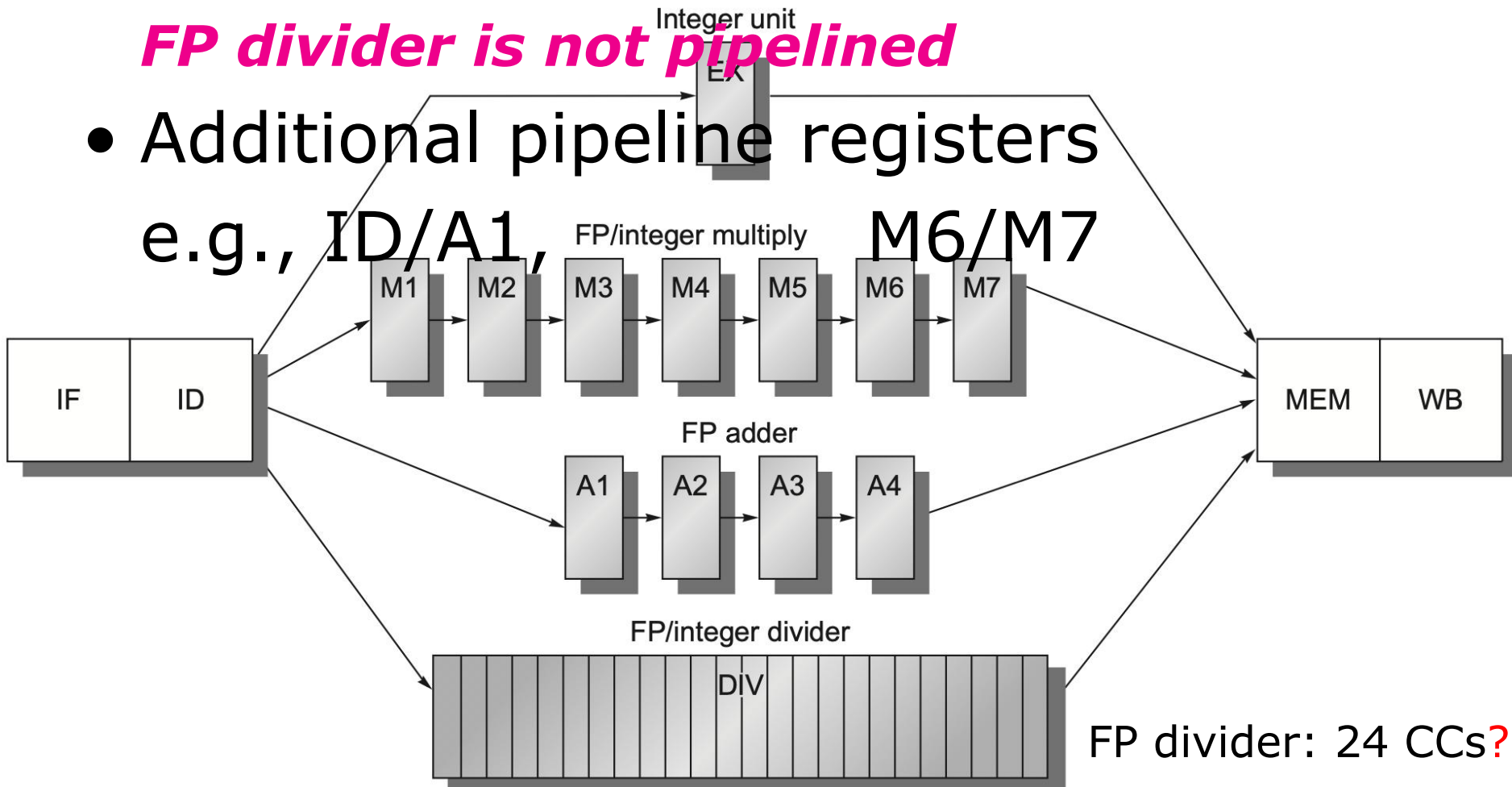
Generalized FP Pipeline

- EX is pipelined (except for FP divider)

FP divider is not pipelined

- Additional pipeline registers

e.g., ID/A1, FP/integer multiply, M6/M7



Generalized FP Pipeline

- Example: independent FP instr
italics: stage where data is needed
bold: stage where a result is available

MUL.D	IF	ID	<u><i>M1</i></u>	M2	M3	M4	M5	M6	<u>M7</u>	MEM	WB
ADD.D		IF	ID	<u><i>A1</i></u>	A2	A3	<u>A4</u>	MEM	WB		
L.D			IF	ID	<u><i>EX</i></u>	<u>MEM</u>	WB				
S.D				IF	ID	<u><i>EX</i></u>	<u>MEM</u>	WB			
compute address						store data					

Generalized FP Pipeline

- Example: independent FP instr
italics: stage where data is needed
bold: stage where a result is available

MUL.D	IF	ID	<u><i>M1</i></u>	M2	M3	M4	M5	M6	<u>M7</u>	MEM	WB
ADD.D		IF	ID	<u><i>A1</i></u>	A2	A3	<u>A4</u>	MEM	WB		
L.D			IF	ID	<u><i>EX</i></u>	<u>MEM</u>	WB				
S.D				IF	ID	<u><i>EX</i></u>	<u>MEM</u>	WB			

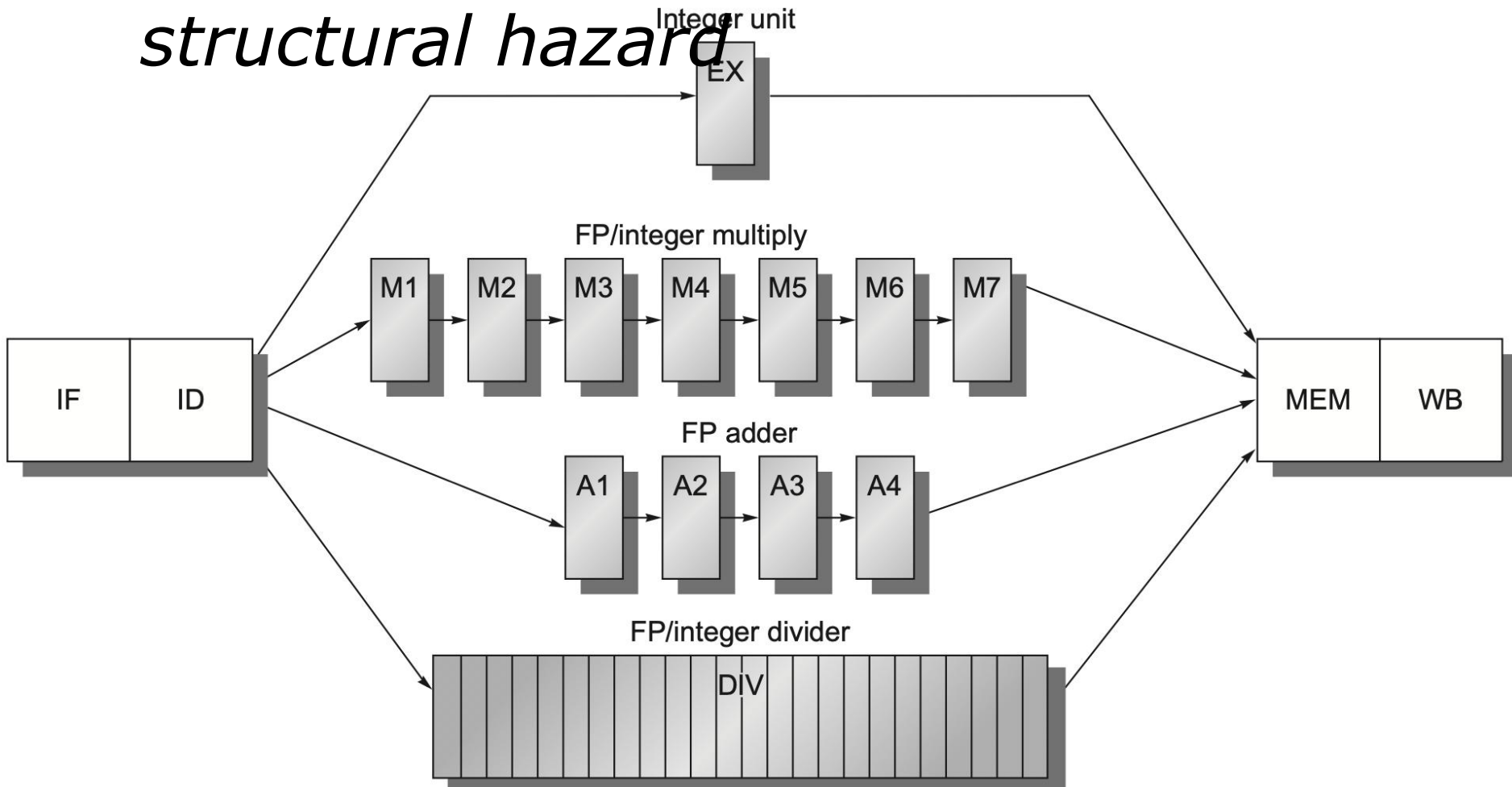
Functional unit	Latency
Integer ALU	0
Data memory (integer and FP loads)	1
FP add	3
FP multiply (also integer multiply)	6

Intervening cycles

Any FP pipeline hazards?

Structural Hazard

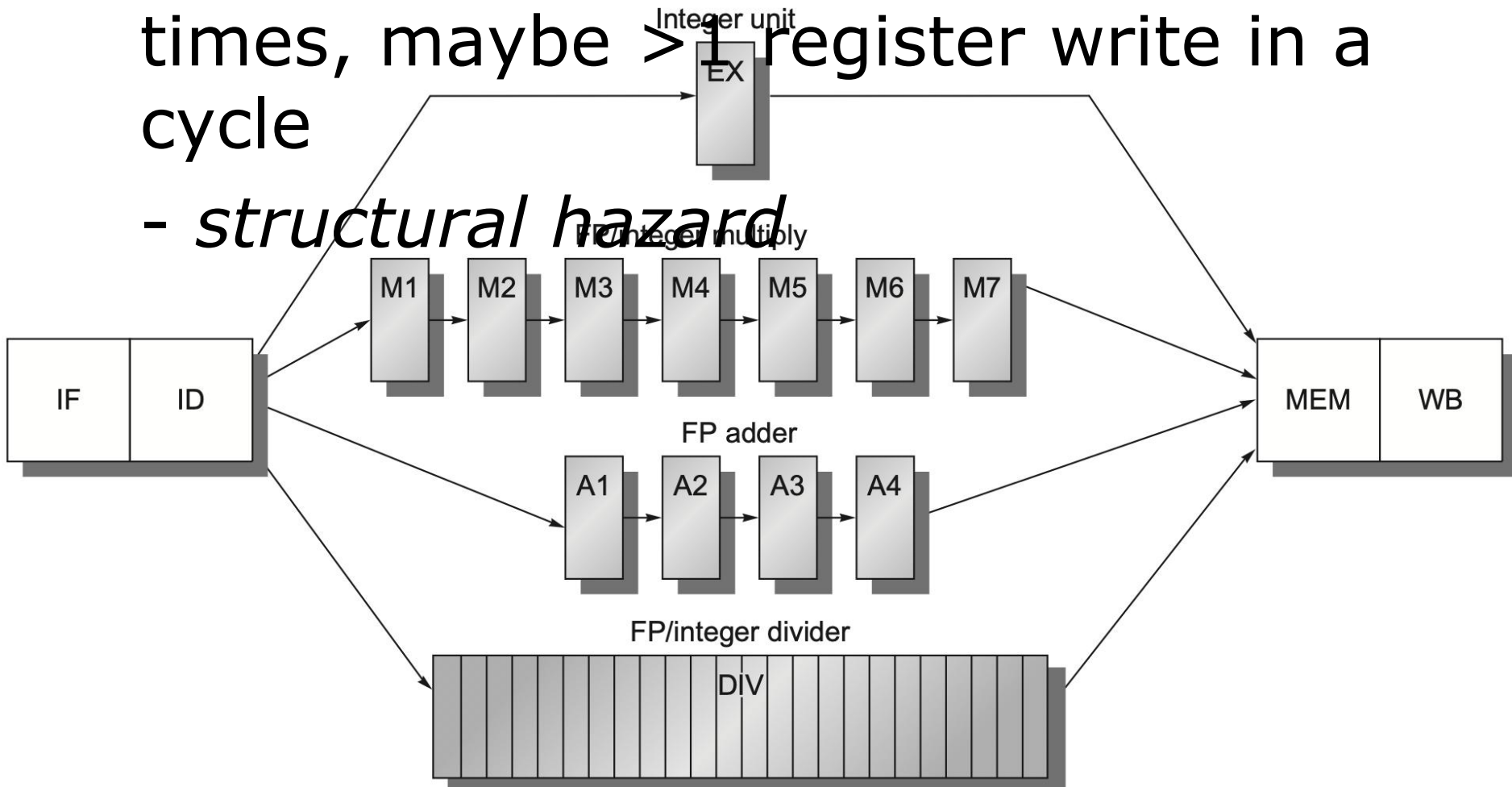
- **Divider** is not fully pipelined – *structural hazard*



Structural Hazard

- Instructions have varying running times, maybe > 1 register write in a cycle

- *structural hazard*



Structural Hazard

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d f2, f4, f6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
fld f2, 0(x2)							IF	ID	EX	MEM	WB

Structural Hazard

- Interlock Detection
- Method 1: track the use of the write port in the ID stage and stall an instruction before it issues
 - ∴ a shift register tracks when already-issued instructions will use the register file;
 - if the instruction in ID needs to use the register file at the same time, stall

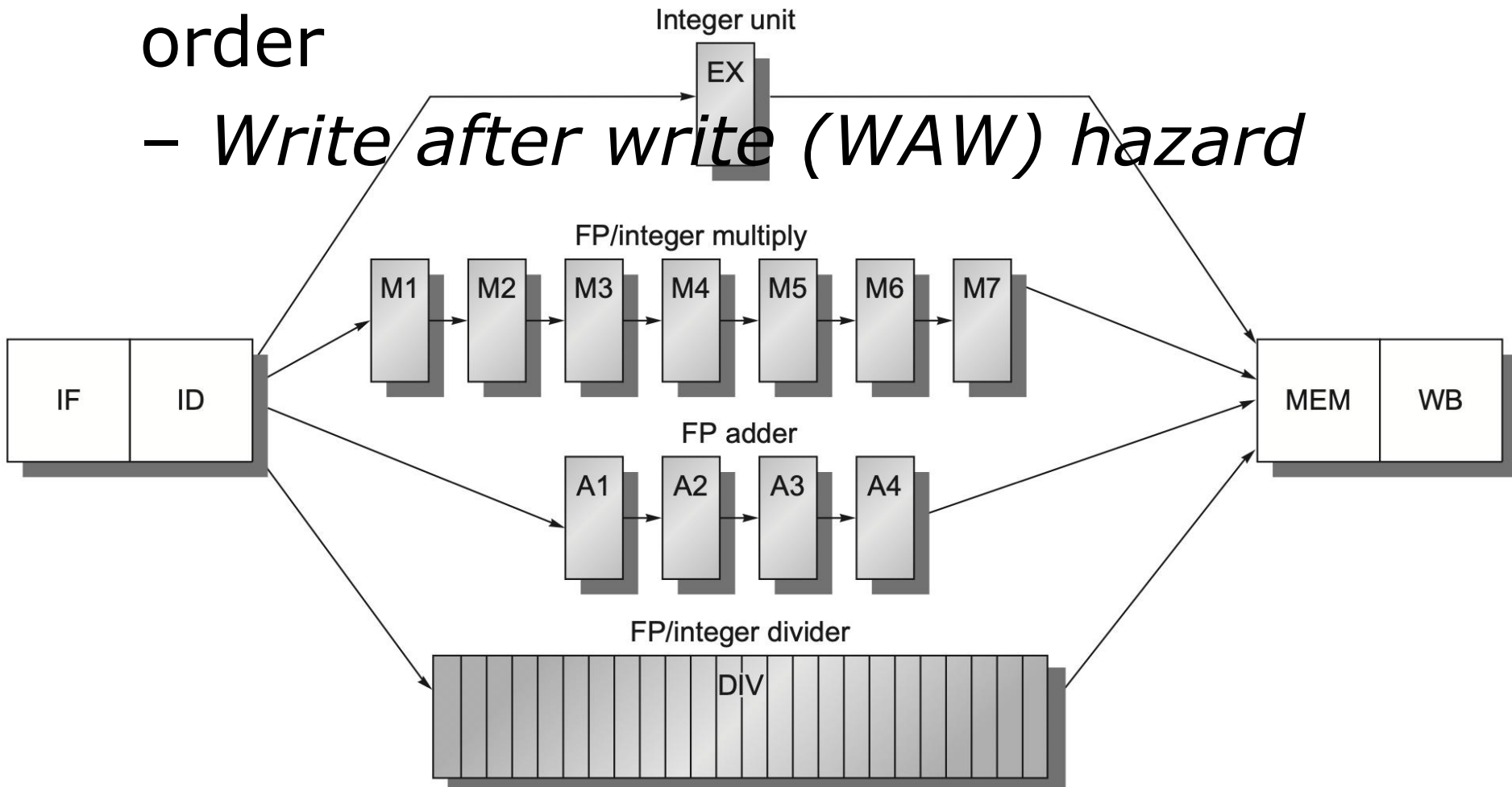
Structural Hazard

- Interlock Detection
- Method 2: stall a conflicting instruction when it tries to enter MEM/WB
 - ∴ could stall either issuing or issued one;
 - give priority to the unit with the longest latency; (*structural hazard, not data hazard*)
 - more complicated*: stall arises from two places - MEM/WB

WAW Hazard

- Instructions no longer reach WB in order

– *Write after write (WAW) hazard*



WAW Hazard

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d <u>f2</u> , f4, f6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
fld <u>f2</u> , 0(x2)							IF	ID	EX	MEM	WB

- If fld were issued one cycle earlier

WAW Hazard

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d <u>f2</u> , f4, f6				IF	ID	A1	A2	A3	A4	MEM	<u>WB</u>
...					IF	ID	EX	MEM	WB		
...											
fld <u>f2</u> , 0(x2)						IF	ID	EX	MEM	<u>WB</u>	

- If fld were issued one cycle earlier
- fld would write F2 one cycle earlier than fadd.d – *WAW hazard*

WAW Hazard

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d <u>f2</u> , f4, f6				IF	ID	A1	A2	A3	A4	MEM	<u>WB</u>
...					IF	ID	EX	MEM	WB		
...											
fld <u>f2</u> , 0(x2)						IF	ID	EX	MEM	<u>WB</u>	

- If fld were issued one cycle earlier
- fld would write F2 one cycle earlier than fadd.d – *WAW hazard*
- Solution 1: delay issuing fld

WAW Hazard

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d <u>f2</u> , f4, f6				IF	ID	A1	A2	A3	A4	MEM	<u>WB</u>
...					IF	ID	EX	MEM	WB		
...											
fld <u>f2</u> , 0(x2)						IF	ID	EX	MEM	<u>WB</u>	

- If fld were issued one cycle earlier
- fld would write F2 one cycle earlier than fadd.d – *WAW hazard*
- Solution 2: zero write control of fadd.d

WAW Hazard

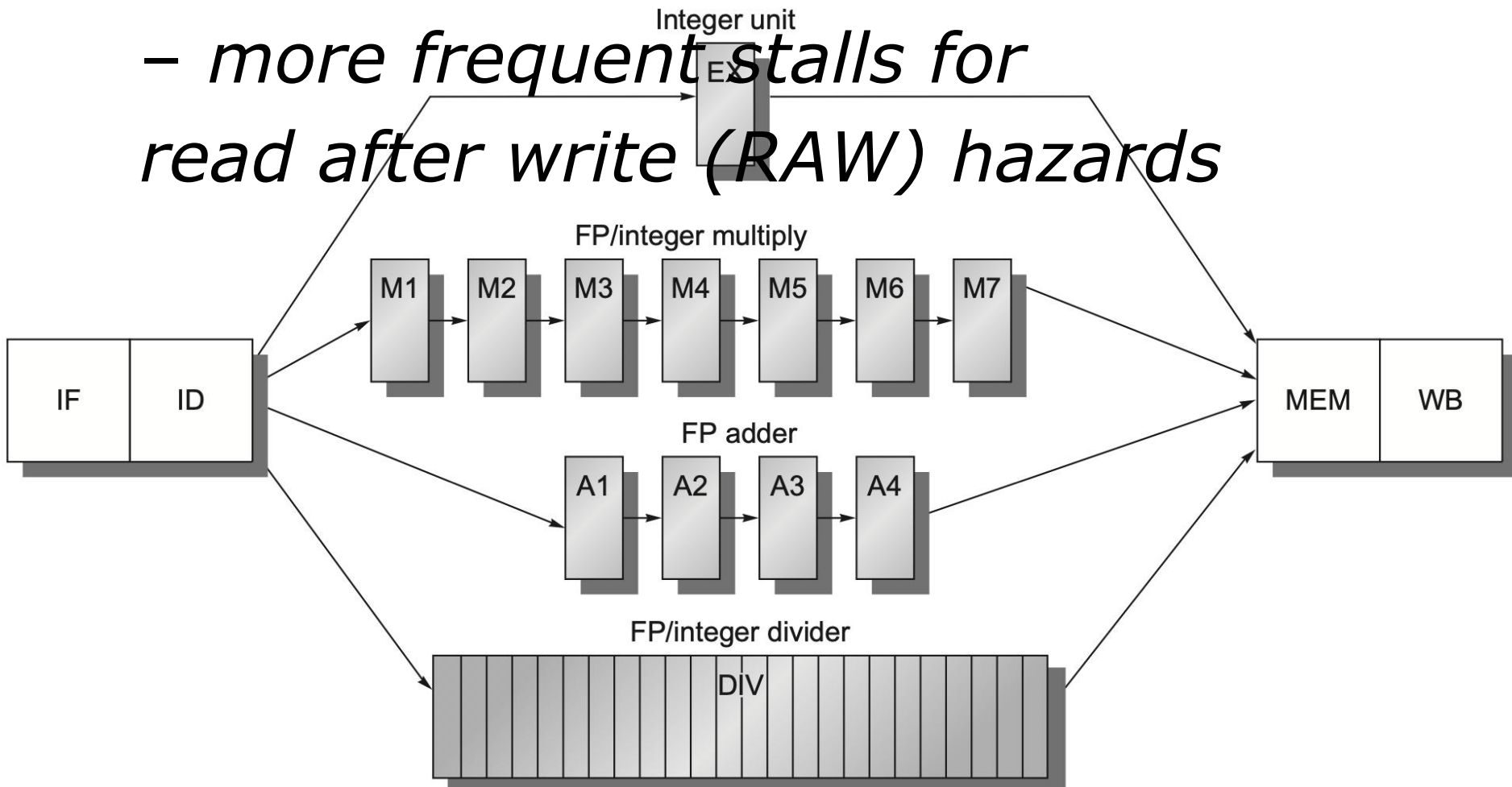
Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0, f4, f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d <u>f2</u> , f4, f6				IF	ID	A1	A2	A3	A4	MEM	<u>WB</u>
...					IF	ID	EX	MEM	WB		
...											
fld <u>f2</u> , 0(x2)						IF	ID	EX	MEM	<u>WB</u>	

- If fld were issued one cycle earlier
- fld would write F2 one cycle earlier than fadd.d – *WAW hazard*
- Solution 2: zero write control of fadd.d
- Make sure that fld decides F2 state
follow program order

RAW Hazard

- Longer latency of operations

– *more frequent stalls for read after write (RAW) hazards*



RAW Hazard

Instruction	Clock cycle number																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
fld f4,0(x2)	IF	ID	EX	MEM	WB												
fmul.d f0, f4, f6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
fadd.d f2, f0, f8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
fsd f2,0(x2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

RAW Hazard

Instruction	Clock cycle number																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
fld f4,0(x2)	IF	ID	EX	MEM	WB												
fmul.d f0, f4, f6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
fadd.d f2, f0, f8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
fsd f2,0(x2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

The fsd must be stalled an extra cycle so that its MEM does not conflict with the fadd.d

RAW Hazard

Instruction	Clock cycle number																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
fld f4,0(x2)	IF	ID	EX	MEM	WB												
fmul.d f0,f4,f6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
fadd.d f2,f0,f8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
fsd f2,0(x2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

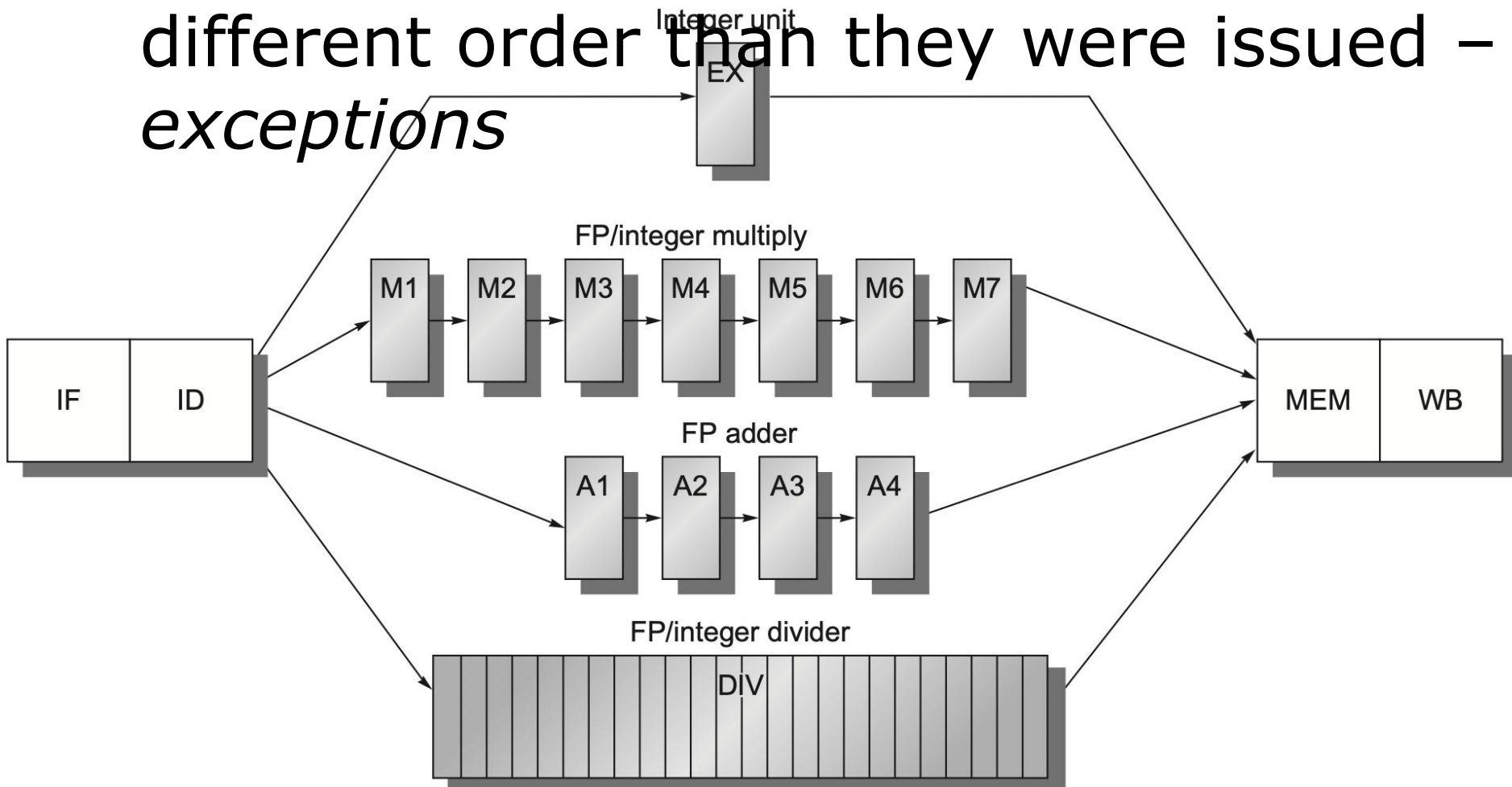
The fsd must be stalled an extra cycle so that its MEM does not conflict with the ADD.D

Instruction	Clock cycle number										
	1	2	3	4	5	6	7	8	9	10	11
fmul.d f0,f4,f6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
fadd.d f2,f4,f6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
fld f2,0(x2)							IF	ID	EX	MEM	WB

Only the fld actually uses the memory
So no structure hazard exists for MEM

Hazard: Exceptions

- Instructions may complete in a different order than they were issued – *exceptions*



**How to detect and solve
pipeline hazards?**

Hazard Detection in ID

- 1. Check for **structural hazards**
wait until the required functional unit is not busy (only for divides);
make sure the register write port is available when it will be needed;

Hazard Detection in ID

- 2. Check for **RAW data hazards**
wait until source registers are available
when needed --- *when they are not
pending destinations of issued
instructions*

Hazard Detection in ID

- 3. Check for **WAW data hazards**
determine if any instruction in A1 – A4,
D, M1-M7 has the same register
destination as this instruction;
if so, stall the issue of the instr in ID

Hazard Detection in ID

- 3. Check for **WAW data hazards**
determine if any instruction in A1 – A4,
D, M1-M7 has the same register
destination as this instruction;
if so, stall the issue of the instr in ID
this is a simplified version;

Hazard Detection in ID

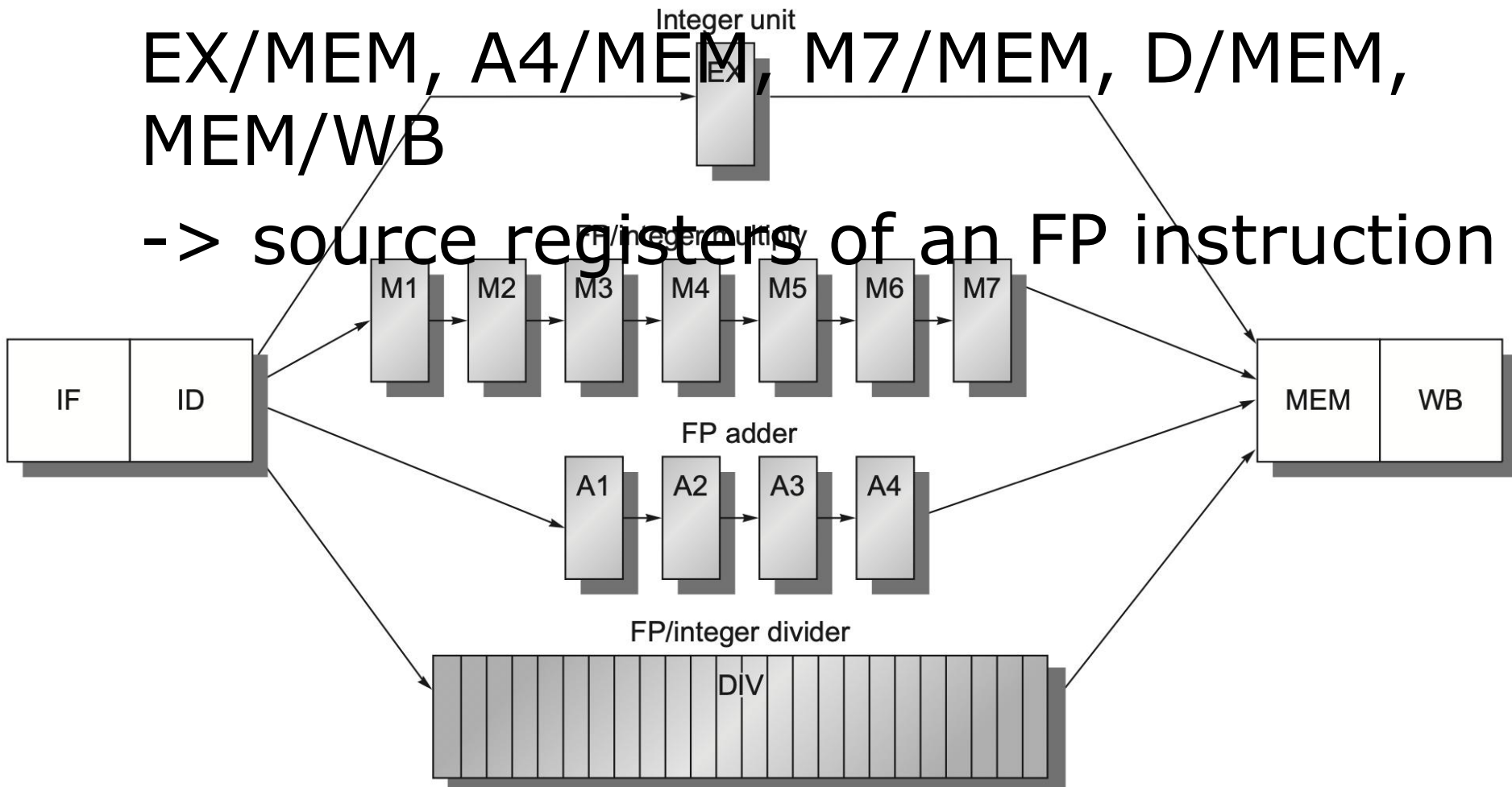
- 3. Check for **WAW data hazards**
determine if any instruction in A1 – A4,
D, M1-M7 has the same register
destination as this instruction;
if so, stall the issue of the instr in ID
this is a simplified version;
more complex: detect instr in ID might
finish before an issued instr, knowing
pipeline length and its current position

Forwarding

- Generalized with more sources

EX/MEM, A4/MEM, M7/MEM, D/MEM,
MEM/WB

-> source registers of an FP instruction



Out-of-Order Completion

```
fdiv.d      f0, f2, f4  
fadd.d      f10, f10, f8  
fsub.d      f12, f12, f14
```

- ADD and SUB complete before DIV
- **Out-of-order completion**: instructions are completing in a different order than they were issued

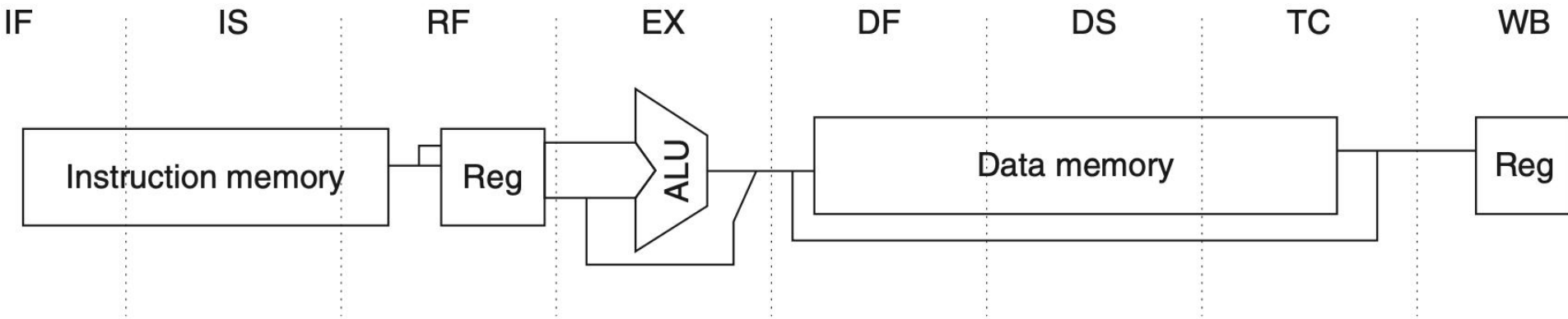
Out-of-Order Completion

How to deal with out-of-order?

- 1. ignore the problem
- 2. buffer the results of an operation until all the operations issued earlier complete
- 3. track what operations were in the pipeline and their PCs for trap-handling
- 4. issue an instruction only if it is certain that all previous instructions will complete without exception

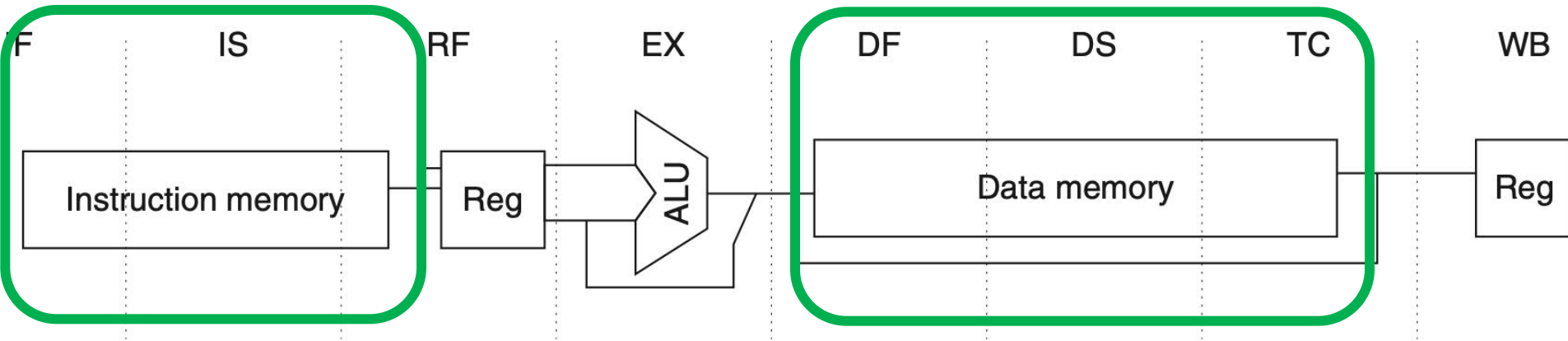
All in MIPS R4000

MIPS R4000



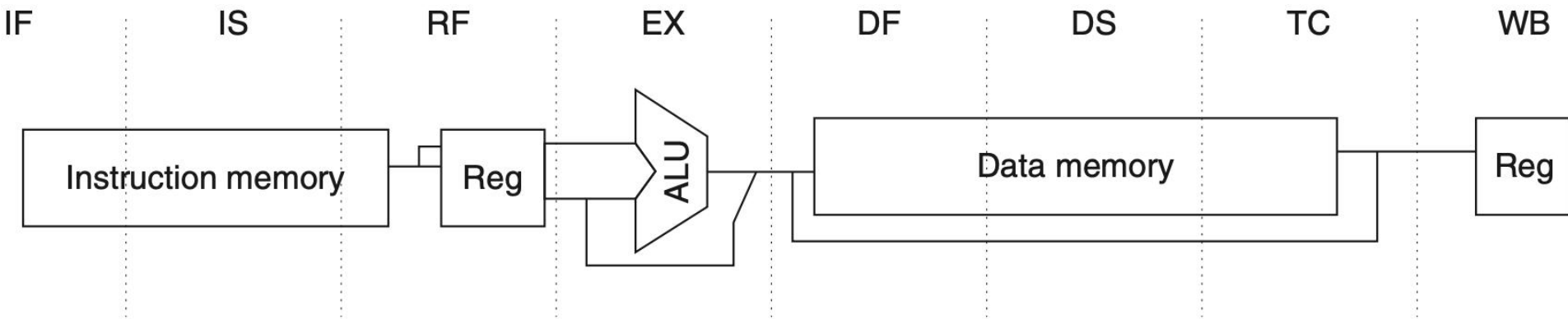
- 5-stage -> 8-stage
- Higher clock rate

MIPS R4000



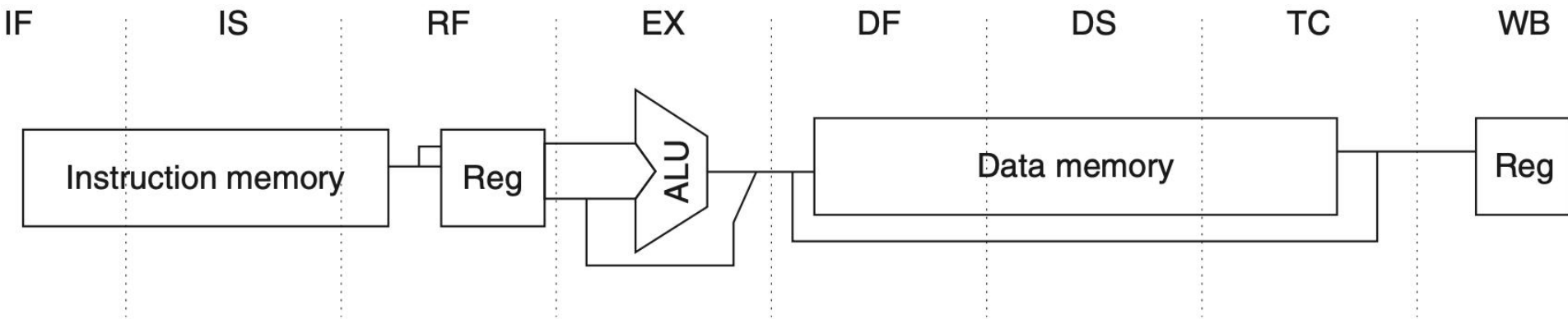
- 5-stage -> 8-stage
- Higher clock rate
- Superpipelining: decompose the memory access because cache access is particularly time critical

MIPS R4000:IF



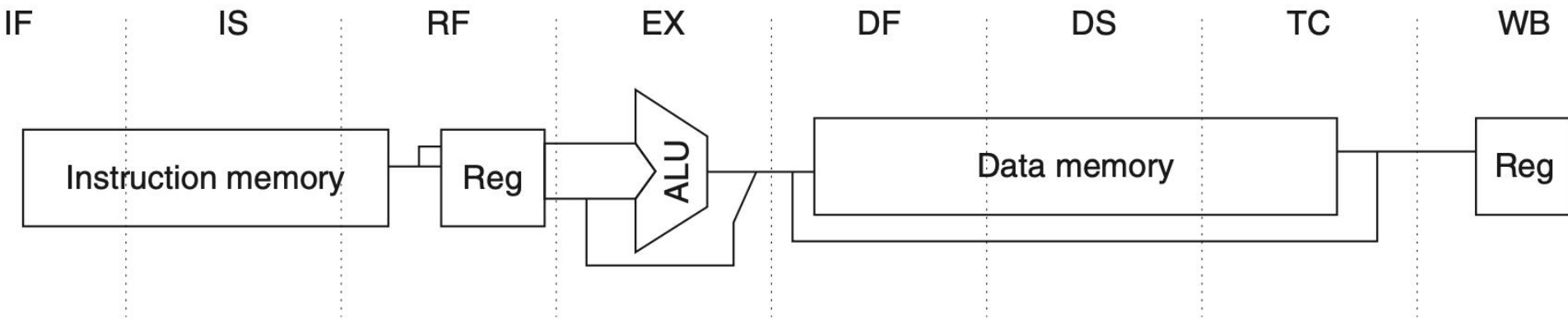
- **IF:** first half of instruction fetch;
PC selection;
initiation of instruction cache access;

MIPS R4000:IS



- **IS:** second half of instruction fetch;
completion of instruction cache access;

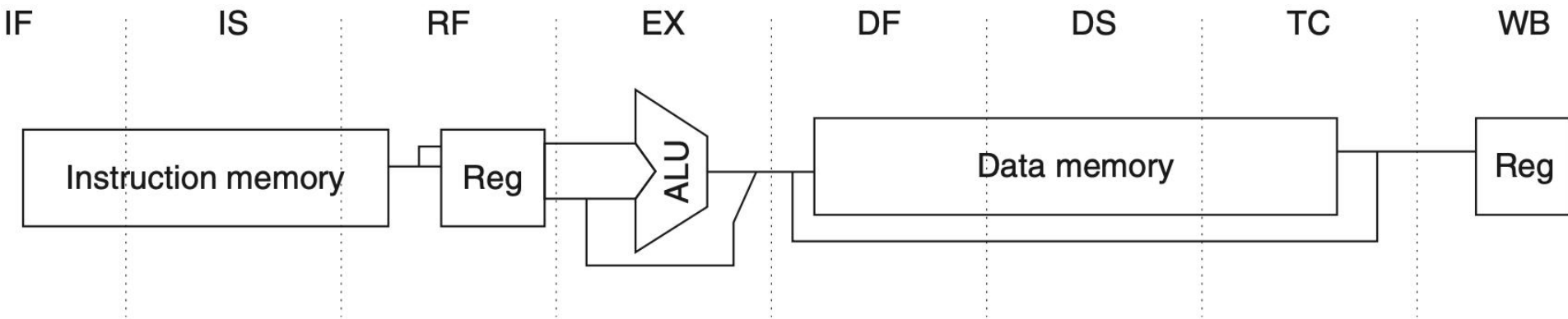
MIPS R4000: RF



- **RF:**

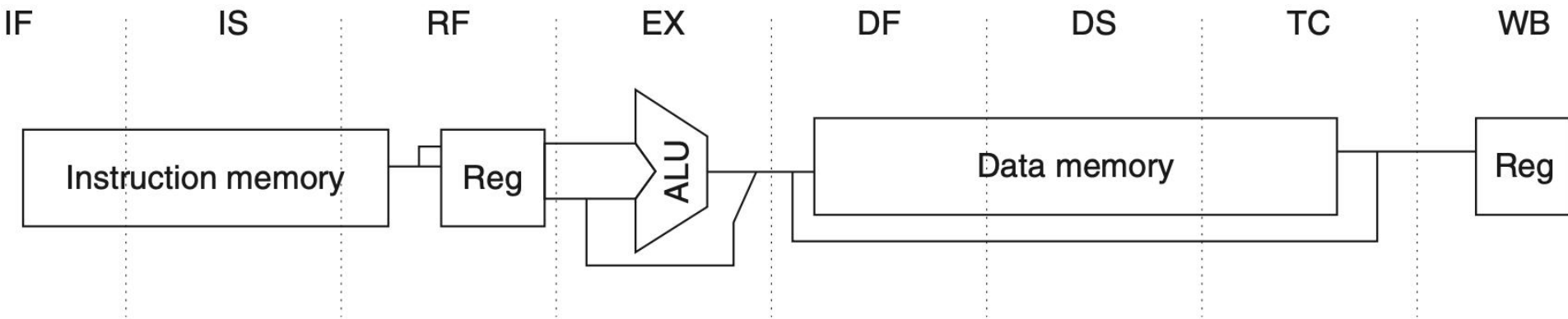
instruction decode and register fetch;
hazard checking;
instruction cache hit detection;

MIPS R4000: EX



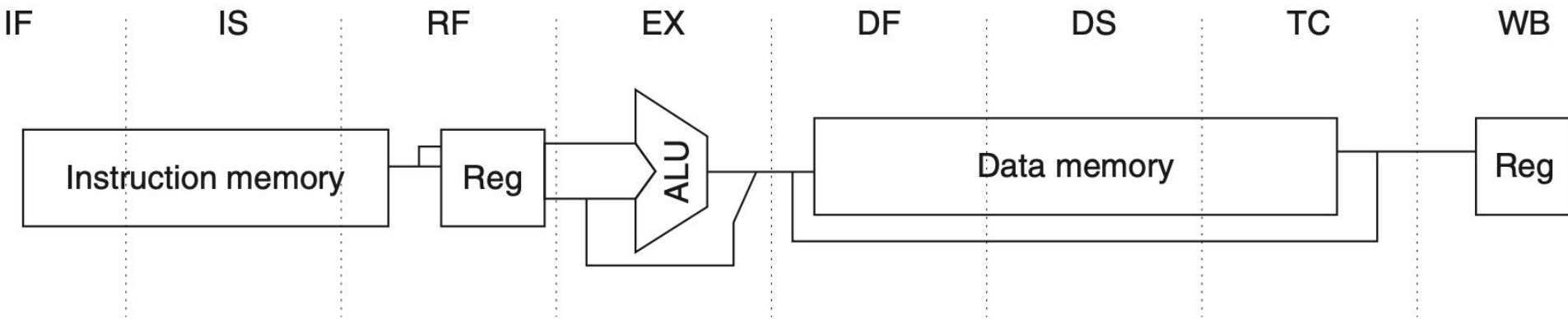
- **EX:** execution
effective address calculation;
ALU operation;
branch-target computation and
condition evaluation;

MIPS R4000:DF



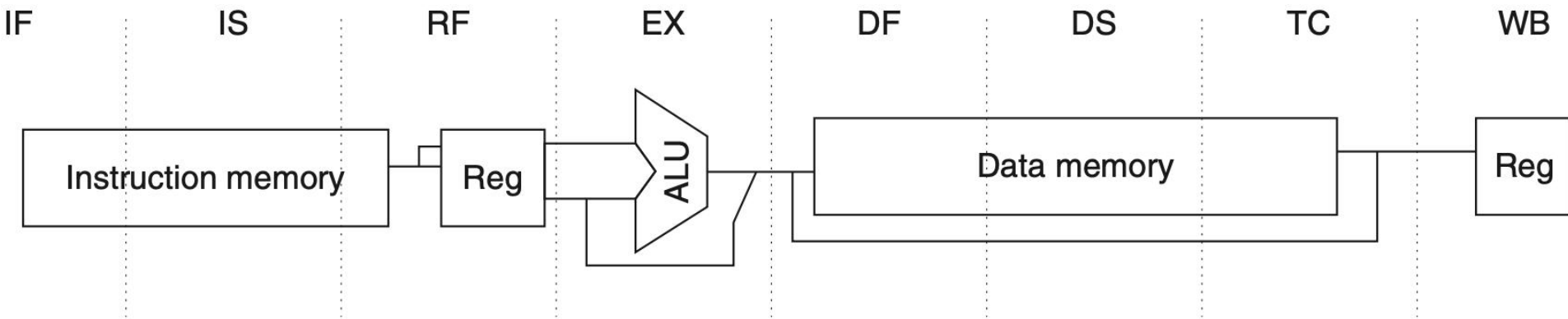
- **DF:** data fetch
first half of data access;

MIPS R4000: DS



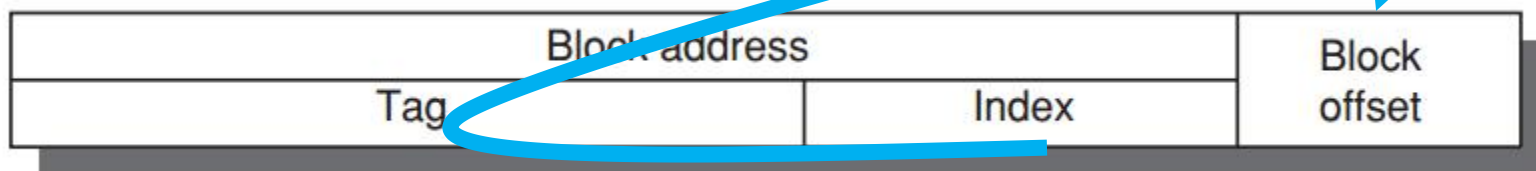
- **DS:** second half of data fetch
completion of data cache access;

MIPS R4000:TC

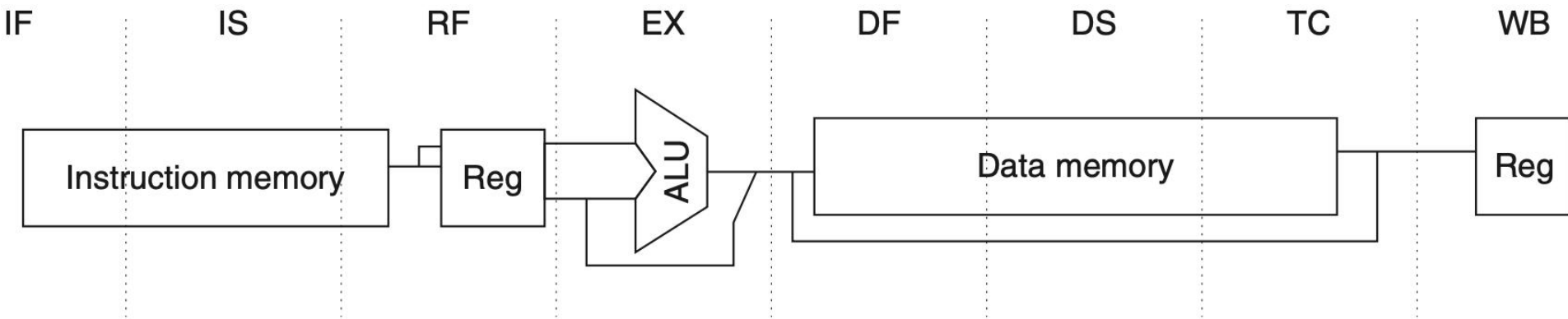


- **TC:** tag check

determine whether the data cache access hit;



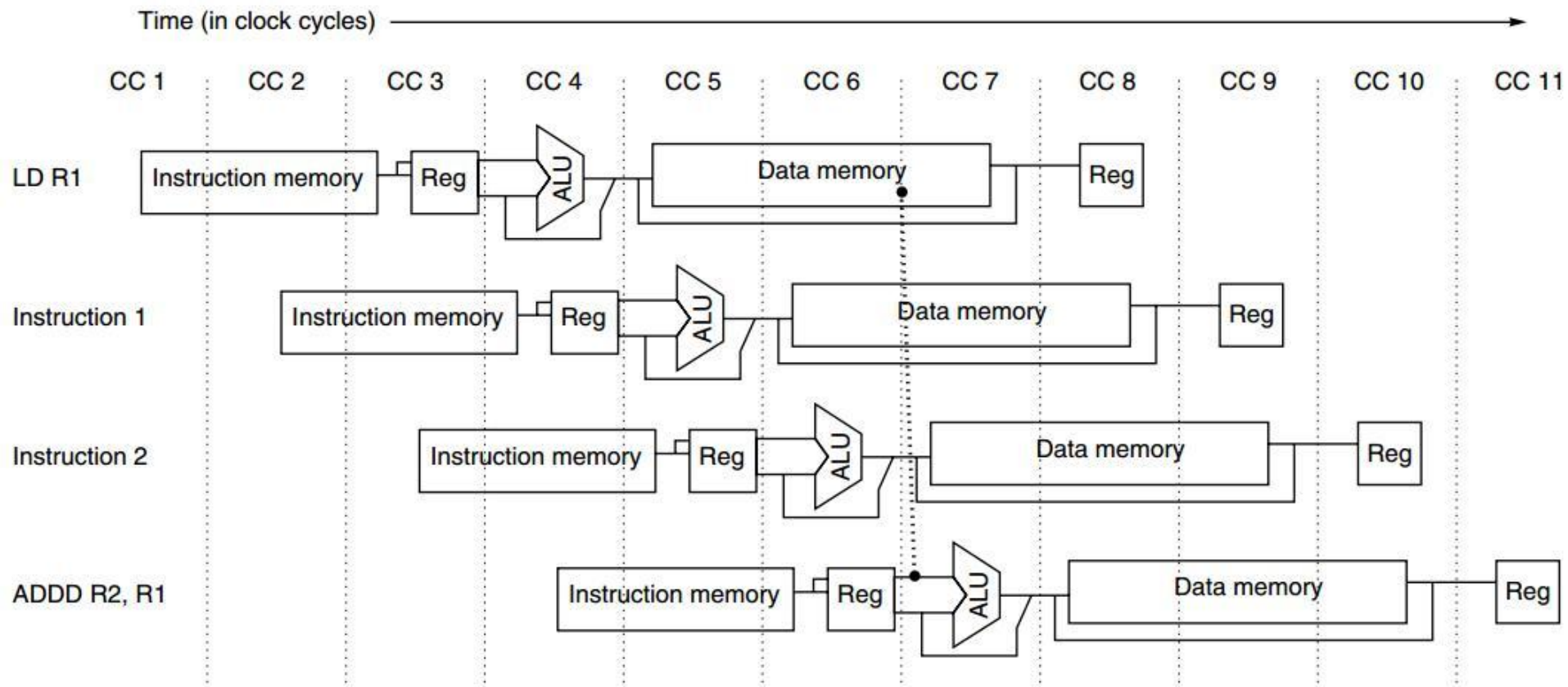
MIPS R4000:WB



- **WB:** write back
for loads and register-register ops;

Load Delay

- 2-cycle load delay
(per the subsequent pipeline diagram)



Load Delay

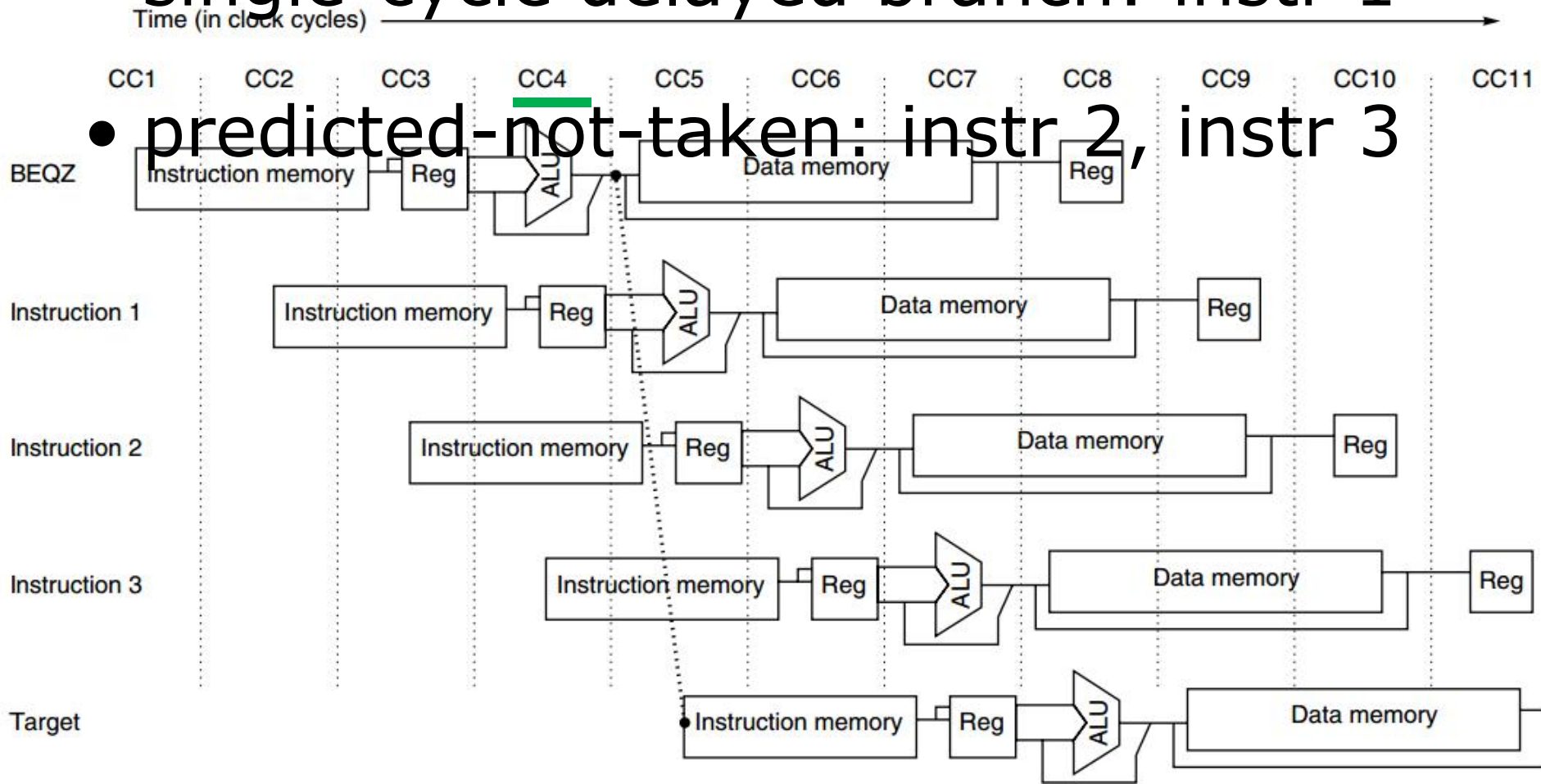
- 2-cycle load delay

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
ld x1,...	IF	IS	RF	EX	DF	DS	TC	WB	
add x2,x1,...		IF	IS	RF	Stall	Stall	EX	DF	DS
sub x3,x1,...			IF	IS	Stall	Stall	RF	EX	DF
or x4,x1,...				IF	Stall	Stall	IS	RF	EX

- **DS:** second half of data fetch
completion of data cache access;

Branch Delay

- 3-cycle branch delay: condition test- EX
- single-cycle delayed branch: instr 1
- ~~predicted-not-taken~~: instr 2, instr 3



Branch Delay

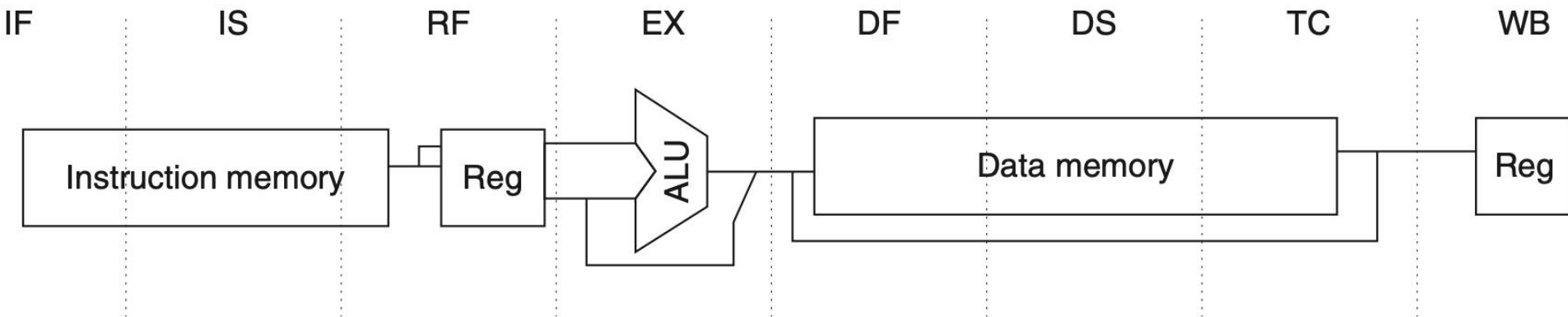
- 3-cycle branch delay: for taken

Instruction number	predicted-not-taken				taken branch				
	1	2	3	4	5	6	7	8	9
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Stall			stall	stall	stall	stall	stall	stall	stall
Stall				stall	stall	stall	stall	stall	stall
Branch target					IF	IS	RF	EX	DF

Instruction number	predicted-not-taken				untaken branch				
	1	2	3	4	5	6	7	8	9
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Branch instruction + 2			IF	IS	RF	EX	DF	DS	TC
Branch instruction + 3				IF	IS	RF	EX	DF	DS

Forwarding

- Forwarding
ALU/MEM or MEM/WB
-> EX/DF, DF/DS, DS/TC, TC/WB



FP Operations

- FP Pipeline
- FP unit with three functional units:
FP divider, FP multiplier, FP adder
- 2 cycles to 112 cycles

Stage vs FP Unit

- FP unit with eight different stages

Stage	Functional unit	Description
A	FP adder	Mantissa add stage
D	FP divider	Divide pipeline stage
E	FP multiplier	Exception test stage
M	FP multiplier	First stage of multiplier
N	FP multiplier	Second stage of multiplier
R	FP adder	Rounding stage
S	FP adder	Operand shift stage
U		Unpack FP numbers

Latency & Ini Interval

- FP operations: latency and initiation interval

FP instruction	Latency	Initiation interval	Pipe stages
Add, subtract	4	3	U, S+A, A+R, R+S
Multiply	8	4	U, E+M, M, M, M, N, N+A, R
Divide	36	35	U, A, R, D^{28} , D+A, D+R, D+A, D+R, A, R
Square root	112	111	U, E, $(A+R)^{108}$, A, R
Negate	2	1	U, S
Absolute value	2	1	U, S
FP compare	3	2	U, A, R

FP Ops: Example 1

- FP multiply + FP add

		Clock cycle											
Operation	Issue/stall	0	1	2	3	4	5	6	7	8	9	10	11 12
Multiply	Issue	U	E + M	M	M	M	N	N + A	R				
Add	Issue		U	S + A	A + R	R + S							
	Issue			U	S + A	A + R	R + S						
	Issue				U	S + A	A + R	R + S					
	Stall					U	S + A	A + R	R + S				
	Stall						U	S + A	A + R	R + S			
	Issue							U	S + A	A + R	R + S		
	Issue								U	S + A	A + R	R + S	

FP Ops: Example 2

- FP add + FP multiply

		Clock cycle												
Operation	Issue/stall	0	1	2	3	4	5	6	7	8	9	10	11	12
Add	Issue	U	S + A	A + R	R + S									
Multiply	Issue		U	E + M	M	M	M	N	N + A	R				
	Issue			U	M	M	M	M	N	N + A	R			

- divide + add

Stall	U	S + A	A + R	R + S					
Stall		U	S + A	A + R	R + S				
Stall			U	S + A	A + R	R + S			
Stall				U	S + A	A + R	R + S		
Stall					U	S + A	A + R	R + S	
Stall						U	S + A	A + R	R + S

FP Ops: Example 4

- FP add + FP divide

Operation	Issue/stall	Clock cycle												
		0	1	2	3	4	5	6	7	8	9	10	11	12
Add	Issue	U	S + A	A + R	R + S									
Divide	Stall		U	A	R	D	D	D	D	D	D	D	D	D
	Issue			U	A	R	D	D	D	D	D	D	D	D
	Issue				U	A	R	D	D	D	D	D	D	D

How to exploit ILP?

ILP Exploitation

- Compiler-based static parallelism
only in domain-specific environments
or in well-structured scientific
applications with significant data-level
parallelism
- Hardware-based dynamic parallelism

How much ILP in a program?

Parallel Instructions

- Two instructions that can execute simultaneously in a pipeline of arbitrary depth without causing any stalls, assuming sufficient resources and thus no structural hazards

Dependent Instructions

- Two instructions that are not parallel
- Must be executed in order
- May often be partially overlapped

Dependence

- Data dependence
- Name dependence
- Control dependence

Data Dependence

An instruction j is data-independent on instruction i if either of the following holds:

- Instruction i produces a result that may be used by instruction j ;
- Instruction j is data-dependent on instruction k , and instruction k is data dependent on i ;

Data Dependence

An instruction j is data-independent on instruction i if either of the following holds:

- Instruction i produces a result that may be used by instruction j ;
- Instruction j is data-dependent on instruction k , and instruction k is data dependent on i ;

(Dependence within a single instr is not considered as a dep: `add x1, x1, x1`)

Data Dependence

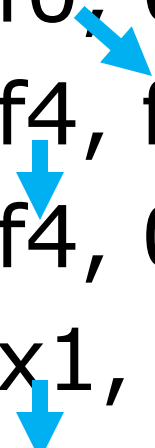
- Example code sequence

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

Data Dependence

- Example code sequence

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```



- Write → Read

Name Dependence

- Two instructions use the same register or memory location (*name*), but there is **no flow of data** between instructions associated w/ that name
- **Read → Write** or **Write → Write**

Name Dependence

An instruction i that precedes instr j in program order:

- **Antidependence: Read $\rightarrow$ Write**
instruction j writes a register or memory location that instruction i reads
- **Output dependence: Write $\rightarrow$ Write**
instruction i and instruction j write the same register or memory location

Name Dependence

- Example code sequence

```
Loop: fld      f0, 0(x1)
      fadd.d   f0, f1, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

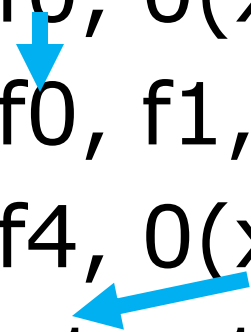


- x1: Read → Write, antidependence

Name Dependence

- Example code sequence

```
Loop: fld      f0, 0(x1)
      fadd.d   f0, f1, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```



- x1: Read → Write, antidependence
- f0: Write → Write, output dependence

Name Dependence

- **Register Renaming:**
change the name (register number or memory location) used in name-dependent instructions such that the instructions do not conflict
- **Statically by compiler or dynamically by hardware**
- Renamed instructions can execute simultaneously or be reordered

Data Hazard

Hazard exists whenever

- 1) there is a name/data dependence between instructions;
- 2) they are close enough; AND
- 3) the overlap during execution would change the order of access to the operand involved in the dependence

Data Hazard

- Read After Write (RAW)
 j reads a source before i writes it,
 j incorrectly gets the old value
- Write After Write (WAW)
 j writes a destination before i writes it,
leaving the value written by i but not j
- Write After Read (WAR)
 j writes a destination before i reads it,
 i incorrectly gets the new value

Control Dependence

- Example code sequence

```
if p1 {  
    S1;  
};  
if p2 {  
    S2;  
};
```

Control Dependence

- Example code sequence

```
if p1 {  
    S1;
```

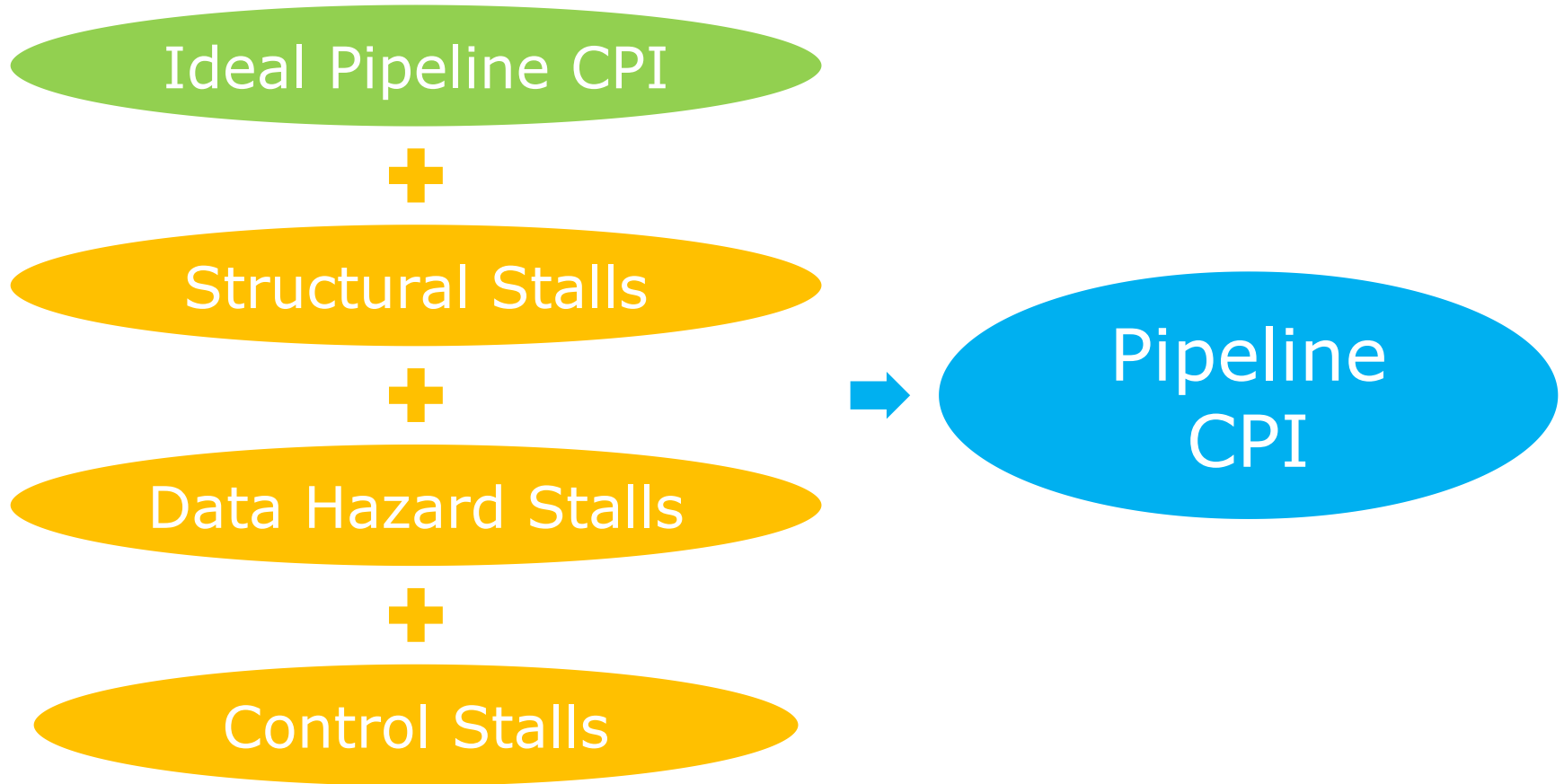
```
};
```

```
if p2 {  
    S2;
```

```
};
```

- S1 is control-dependent on p1
- S2 is control-dependent on p2

Pipelining Performance



Pipelining Optimization

Technique	Reduces	Section
Forwarding and bypassing	Potential data hazard stalls	C.2
Simple branch scheduling and prediction	Control hazard stalls	C.2
Basic compiler pipeline scheduling	Data hazard stalls	C.2, 3.2
Basic dynamic scheduling (scoreboarding)	Data hazard stalls from true dependences	C.7
Loop unrolling	Control hazard stalls	3.2
Advanced branch prediction	Control stalls	3.3
Dynamic scheduling with renaming	Stalls from data hazards, output dependences, and antidependences	3.4
Hardware speculation	Data hazard and control hazard stalls	3.6
Dynamic memory disambiguation	Data hazard stalls with memory	3.6
Issuing multiple instructions per cycle	Ideal CPI	3.7 , 3.8
Compiler dependence analysis, software pipelining, trace scheduling	Ideal CPI, data hazard stalls	H.2, H.3
Hardware support for compiler speculation	Ideal CPI, data hazard stalls, branch hazard stalls	H.4, H.5

Property Constraints

- Exception Behavior

any changes in the ordering of instruction execution must not change how exceptions are raised in the program

- Data Flow

the actual flow of data values among instructions that produce results and those that consume them

Exception Behavior

- Exception Behavior

any changes in the ordering of instruction execution must not change how exceptions are raised in the program;

often relaxed to: reordering of instruction execution must not cause any new exceptions in the program

Exception Behavior

- Example code sequence

add x2, x3, x4

beq x2, x0, L1

ld x1, 0(x2)

L1: ...

Exception Behavior

- Example code sequence

```
add    x2, x3, x4  
        
beq    x2, x0, L1  
ld     x1, 0(x2)
```

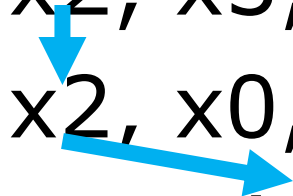
L1: ...

- x2: data dependence

Exception Behavior

- Example code sequence

```
add    x2, x3, x4  
beq    x2, x0, L1  
ld     x1, 0(x2)
```



The diagram illustrates dependencies between instructions. A blue arrow points from the `x2` register in the `add` instruction to the `x2` register in the `beq` instruction, representing a data dependence. Another blue arrow points from the `beq` instruction to the `ld` instruction, representing a control dependence.

L1: ...

- `x2`: `add` → `beq`, data dependence
- `x2`: `beq` → `ld`, control dependence for memory protection

Data Flow

- Data Flow

the actual flow of data values among instructions that produce results and those that consume them;

branches make data flow dynamic as they allow the source of data for a given instruction to come from many points

Data Flow

- Example code sequence

add x1, x2, x3

beq x4, x0, L

sub x1, x5, x6

L: ...

or x7, x1, x8

Data Flow

- Example code sequence

add **x1**, x2, x3

beq x4, x0, L

sub **x1**, x5, x6

L: ...

or x7, **x1**, x8

- **x1** of or instruction is data-dependent on both add and sub instructions
- value of **x1** is control-dependent on beq

How to exploit ILP?

ILP Exploitation

- Compiler-based static parallelism
only in domain-specific environments
or in well-structured scientific
applications with significant data-level
parallelism
- Hardware-based dynamic parallelism

Static Scheduling

- Pipeline Scheduling

re-order instructions to avoid stalls

- Loop Unrolling

replicate the loop body multiple times to increase the number of instructions relative to the branch and overhead instructions

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

highest-address element
of x[i]

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

highest-address element of x[i]

scalar value s

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)  highest-address element  
      fadd.d   f4, f0, f2  of x[i]  
      fsd      f4, 0(x1)  scalar value s  
      addi     x1, x1, -8  decrement pointer per  
      bne      x1, x2, Loop double word
```

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)  highest-address element  
      fadd.d   f4, f0, f2  of x[i]  
      fsd      f4, 0(x1)  scalar value s  
      addi     x1, x1, -8  move to next element  
      bne      x1, x2, Loop
```

Pipeline Scheduling

- Example code segment

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- RISC-V code

```
Loop: fld      f0, 0(x1)  highest-address element of x[i]
      fadd.d   f4, f0, f2  scalar value s
      fsd      f4, 0(x1)
      addi     x1, x1, -8  move to next element
      bne      x1, x2, Loop  Regs[x2]+8 for lowest-address element
```

Pipeline Scheduling

- RISC-V code

```
Loop: fld      f0, 0(x1)
      fadd.d   f4, f0, f2
      fsd      f4, 0(x1)
      addi     x1, x1, -8
      bne      x1, x2, Loop
```

- How many clock cycles per loop?

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

- RISC-V code

Loop: fld f0, 0(x1)
fadd.d f4, f0, f2
fsd f4, 0(x1)
addi x1, x1, -8
bne x1, x2, Loop

1-CC stall

- How many clock cycles per loop?

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

- RISC-V code

Loop: fld f0, 0(x1)

fadd.d f4, f0, f2

1-CC stall

fsd f4, 0(x1)

2-CC stall

addi x1, x1, -8

bne x1, x2, Loop

- How many clock cycles per loop?

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

- RISC-V code

Loop: fld f0, 0(x1)

fadd.d f4, f0, f2

1-CC stall

fsd f4, 0(x1)

2-CC stall

addi x1, x1, -8

bne x1, x2, Loop

- How many clock cycles per loop?: 8

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

- RISC-V code

```
Loop: fld      f0, 0(x1)
      addi     x1, x1, -8
      fadd.d   f4, f0, f2
      fsd      f4, 8(x1)
      bne      x1, x2, Loop
```

2-CC stall

- How many clock cycles per loop?: 7

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

- RISC-V code

```
Loop: fld      f0, 0(x1)
      addi     x1, x1, -8
      fadd.d   f4, f0, f2
      fsd      f4, 8(x1)
      bne      x1, x2, Loop
```

2-CC stall

- How many clock cycles per loop?: 7

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

3/7 is actual work

Pipeline Scheduling

- RISC-V code

Loop: fld f0, 0(x1)
 addi x1, x1, -8
 fadd.d f4, f0, f2
 fsd f4, 8(x1)
 bne x1, x2, Loop

loop overhead

2-CC stall

- How many clock cycles per loop?: 7

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

3/7 is actual work

How to ↓ loop overhead?

Loop Unrolling

- RISC-V code

```
Loop: fld      f0, 0(x1)
      addi     x1, x1, -8
      fadd.d   f4, f0, f2
      fsd      f4, 8(x1)
      bne      x1, x2, Loop
```

2-CC stall

- Replicate **loop body** multiple times
given the same amount of **overhead**
instructions

Loop Unrolling

- RISC-V code

```
Loop: fld      f0,0(x1)
      fadd.d   f4,f0,f2
      fsd      f4,0(x1)          //drop addi & bne
      fld      f6,-8(x1)
      fadd.d   f8,f6,f2
      fsd      f8,-8(x1)        //drop addi & bne
      fld      f0,-16(x1)
      fadd.d   f12,f0,f2
      fsd      f12,-16(x1)      //drop addi & bne
      fld      f14,-24(x1)
      fadd.d   f16,f14,f2
      fsd      f16,-24(x1)
      addi     x1,x1,-32
      bne      x1,x2,Loop
```

Loop Unrolling

- RISC-V code

```
Loop: fld      f0,0(x1)  highest-address element  
      fadd.d   f4,f0,f2  of x[i]  
      fsd      f4,0(x1)  //drop addi & bne  
      fld      f6,-8(x1)  
      fadd.d   f8,f6,f2  
      fsd      f8,-8(x1) //drop addi & bne  
      fld      f0,-16(x1)  
      fadd.d   f12,f0,f2  
      fsd      f12,-16(x1) //drop addi & bne  
      fld      f14,-24(x1)  
      fadd.d   f16,f14,f2  
      fsd      f16,-24(x1)  
      addi     x1,x1,-32  
      bne     x1,x2,Loop  Regs[x2]+32  
                          for lowest-address element
```

Loop Unrolling

- RISC-V code: 26-CC per four ops

```
Loop: fld      f0,0(x1)  highest-address element  
      fadd.d   f4,f0,f2  of x[i]  
      fsd      f4,0(x1)  //drop addi & bne  
      fld      f6,-8(x1)  
      fadd.d   f8,f6,f2  
      fsd      f8,-8(x1) //drop addi & bne  
      fld      f0,-16(x1)  
      fadd.d   f12,f0,f2  
      fsd      f12,-16(x1) //drop addi & bne  
      fld      f14,-24(x1)  
      fadd.d   f16,f14,f2  
      fsd      f16,-24(x1)  
      addi     x1,x1,-32  
      bne      x1,x2,Loop  Regs[x2]+32  
                           for lowest-address element
```

Loop Unrolling

- RISC-V code: 6.5-CC per op

```
Loop: fld      f0,0(x1)
      fadd.d   f4,f0,f2
      fsd      f4,0(x1)           //drop addi & bne
      fld      f6,-8(x1)
      fadd.d   f8,f6,f2
      fsd      f8,-8(x1)         //drop addi & bne
      fld      f0,-16(x1)
      fadd.d   f12,f0,f2
      fsd      f12,-16(x1)       //drop addi & bne
      fld      f14,-24(x1)
      fadd.d   f16,f14,f2
      fsd      f16,-24(x1)
      addi     x1,x1,-32
      bne     x1,x2,Loop
```

highest-address element of x[i]

Regs[x2]+32
for lowest-address element

How to gain more?

Scheduling!

re-order coverage

original: 6 instructions

unrolled: 14 instructions

Loop Unrolling & Scheduling

- RISC-V code: 14-CC per four ops

Loop: fld f0, 0(x1) highest-address element
 fld f6, -8(x1) of x[i]
 fld f0, -16(x1)
 fld f14, -24(x1)
 fadd.d f4, f0, f2
 fadd.d f8, f6, f2
 fadd.d f12, f0, f2
 fadd.d f16, f14, f2
 fsd f4, 0(x1)
 fsd f8, -8(x1)
 fsd f12, 16(x1)
 fsd f16, 8(x1)
 addi x1, x1, -32
 bne x1, x2, Loop Regs[x2]+32
 for lowest-address element

Loop Unrolling & Scheduling

- RISC-V code: 3.5-CC per op

```
Loop:  fld      f0, 0(x1)
        fld      f6, -8(x1)
        fld      f0, -16(x1)
        fld      f14, -24(x1)
        fadd.d    f4, f0, f2
        fadd.d    f8, f6, f2
        fadd.d    f12, f0, f2
        fadd.d    f16, f14, f2
        fsd      f4, 0(x1)
        fsd      f8, -8(x1)
        fsd      f12, 16(x1)
        fsd      f16, 8(x1)
        addi     x1, x1, -32
        bne      x1, x2, Loop
```

highest-address element of x[i]

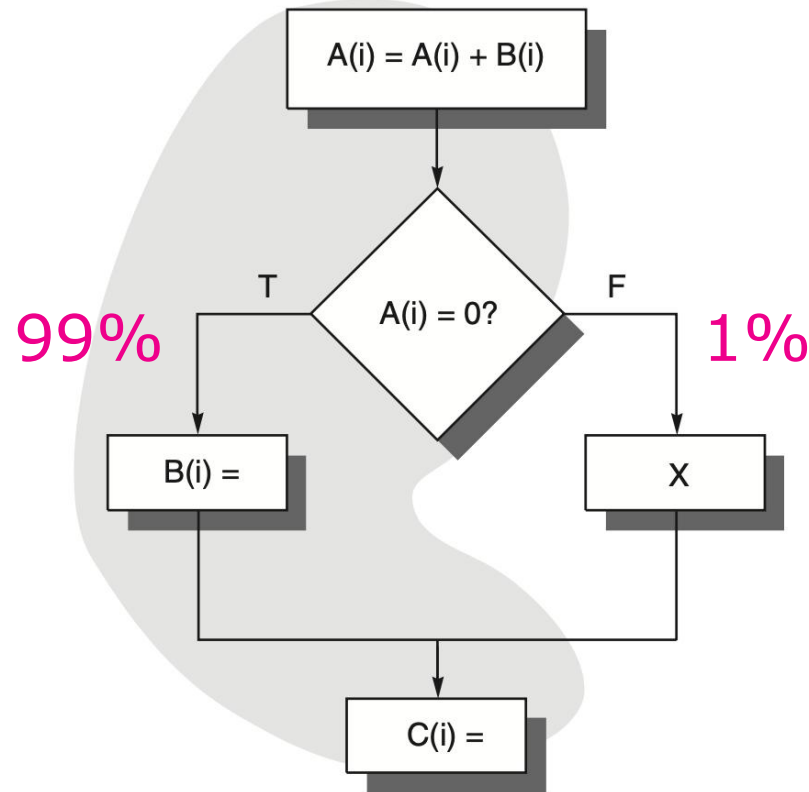
Regs[x2]+32
for lowest-address element

How to handle branches?

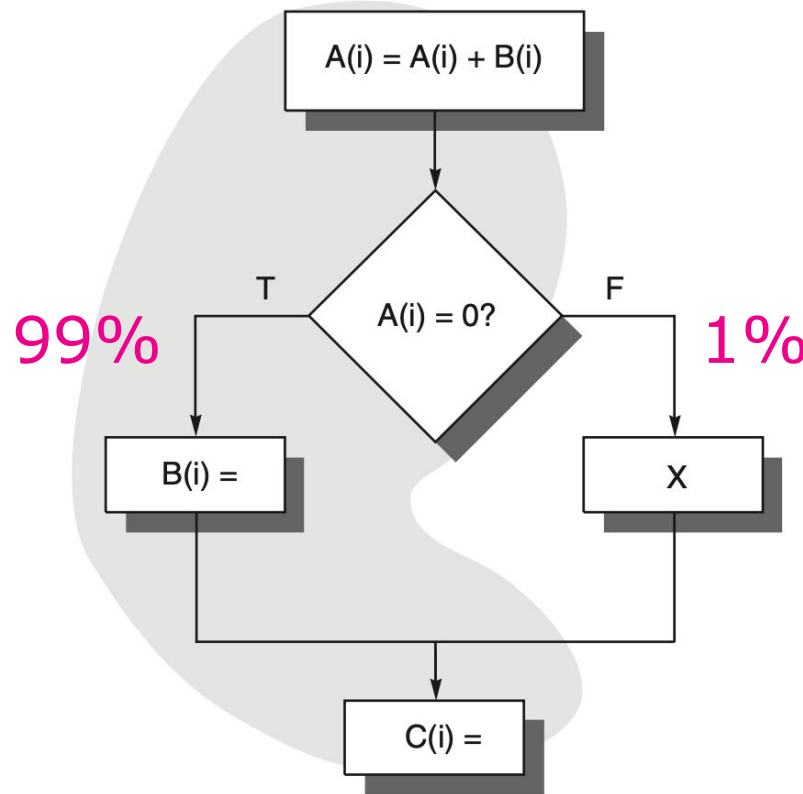
How to handle branches?

often you don't have to

Frequent Critical Path



Frequent Critical Path



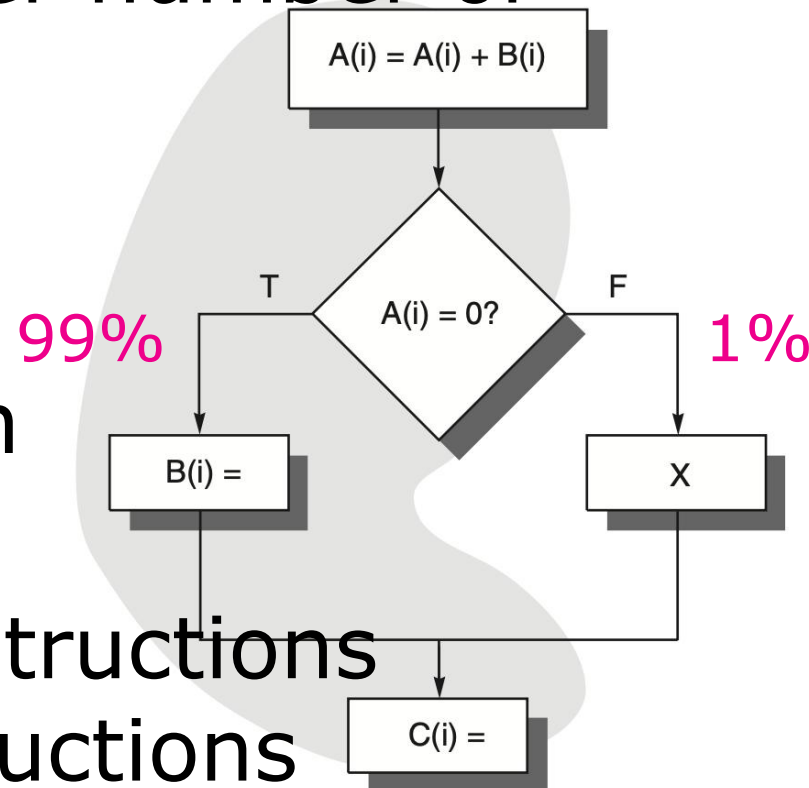
- Focus on a frequent critical path as if the branch were gone

Trace Scheduling

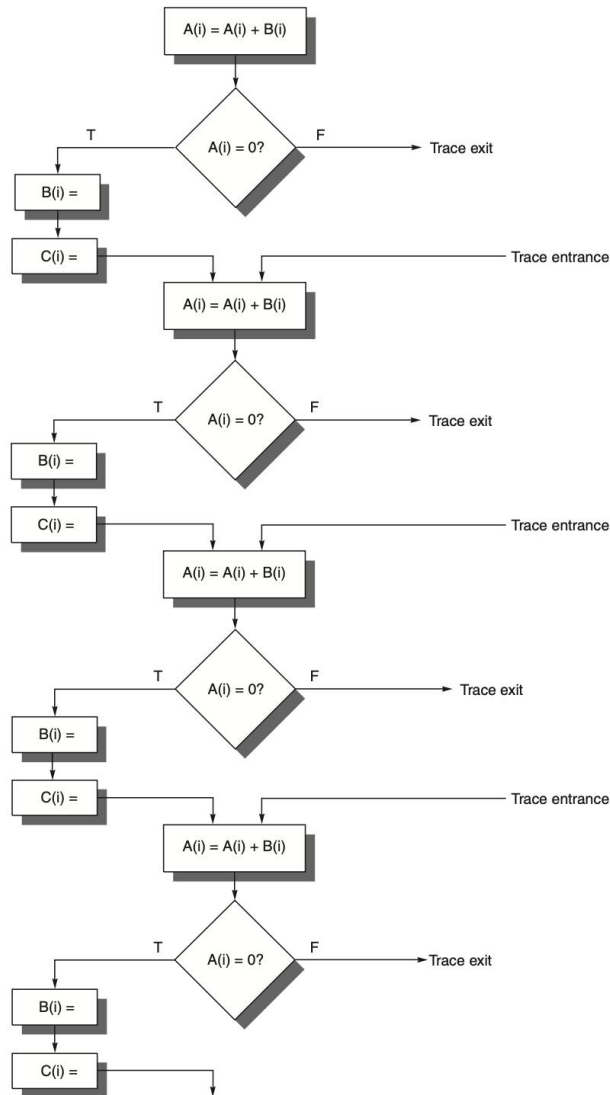
- Trace: a likely sequence of basic blocks whose operations will be put together into a smaller number of instructions

- Trace Selection:
find the frequent path

- Trace Compaction:
compress on-path instructions into fewer wider instructions



Trace Scheduling



- unrolled trace

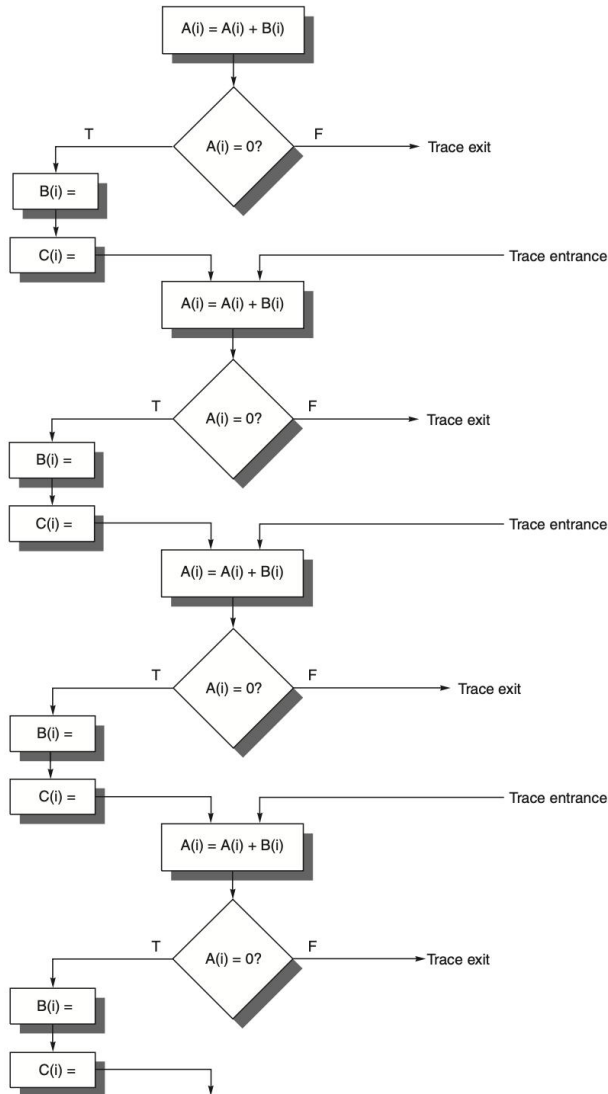
- trace exit

where control flow jumps off the frequent path

- trace entrance

where control flow returns to the trace

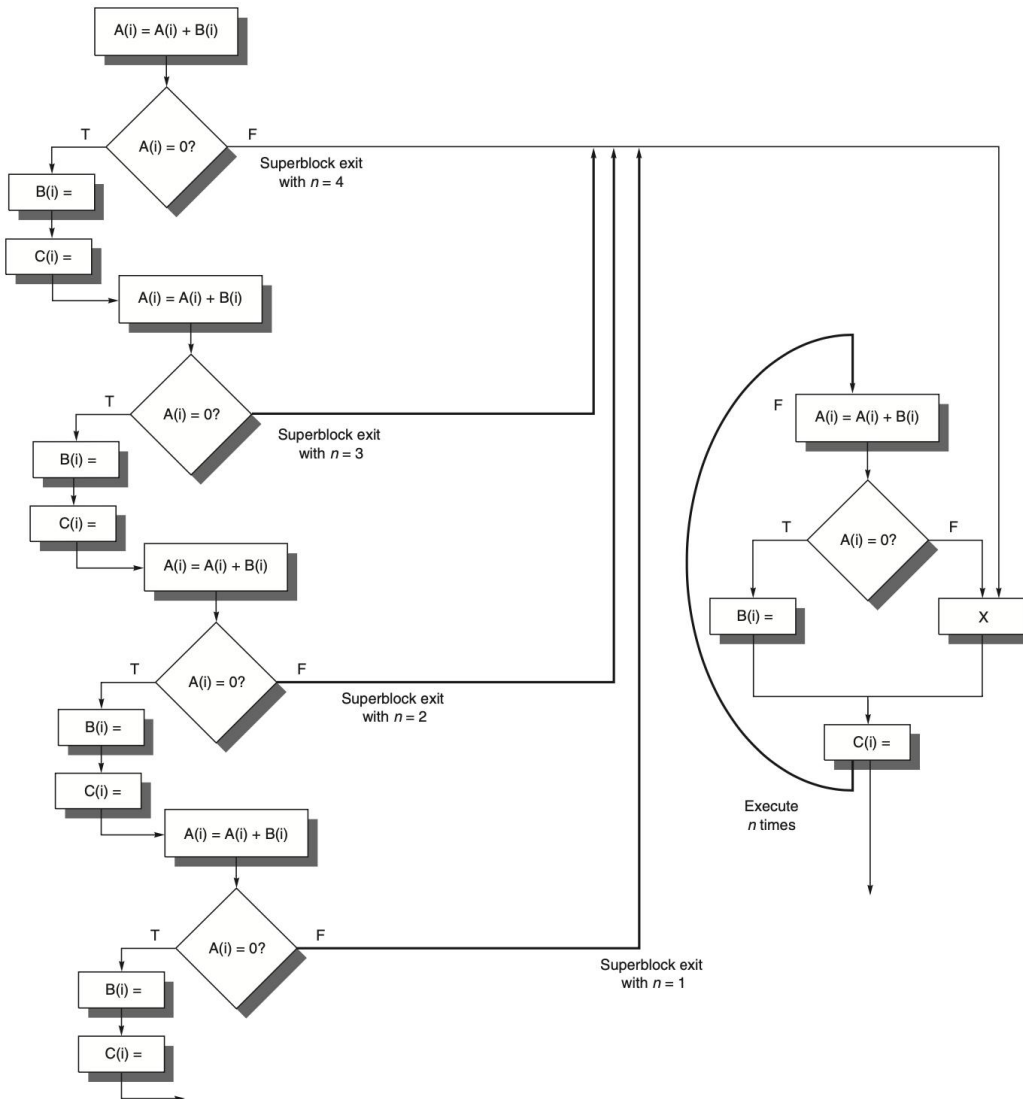
Trace Scheduling



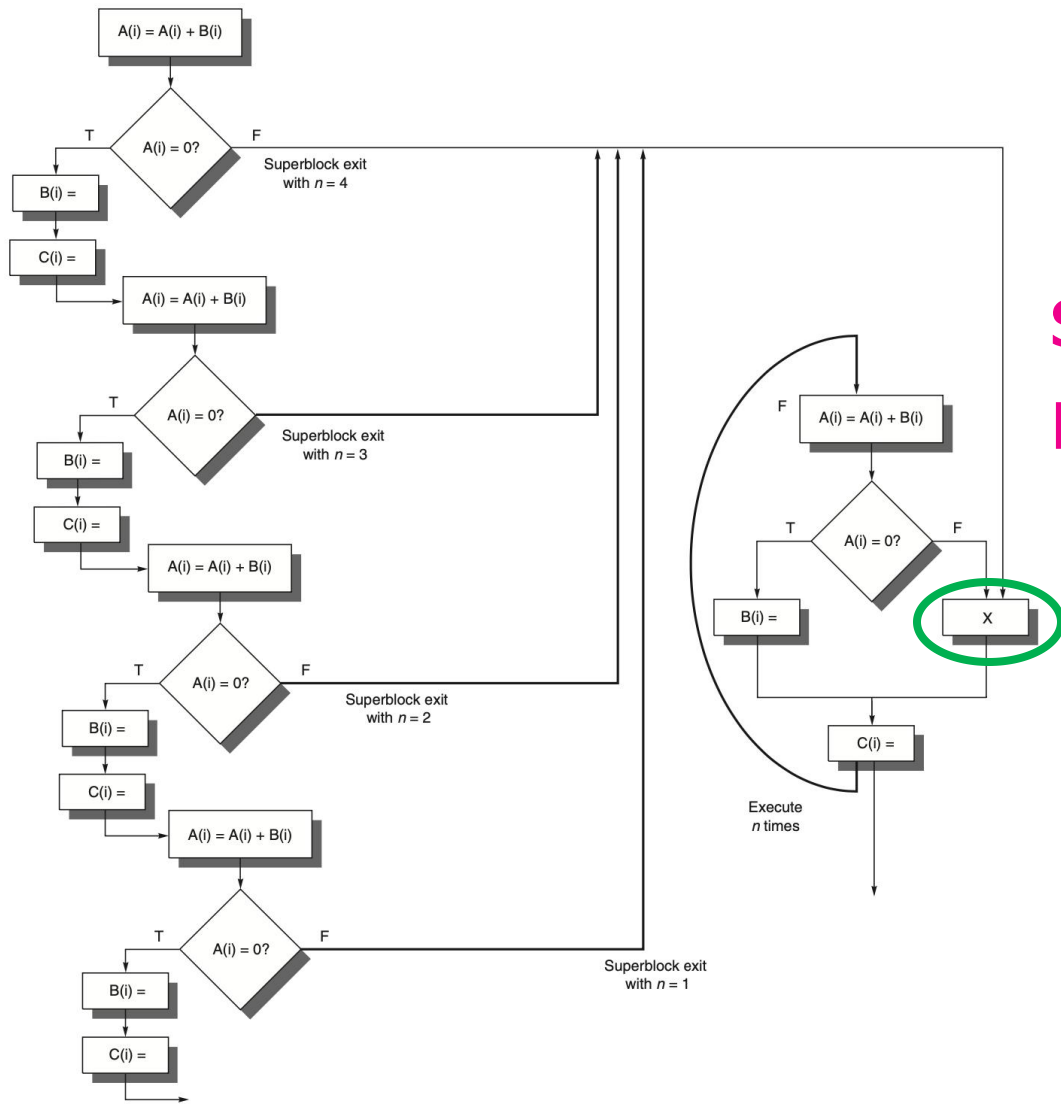
too many entrances and exits amidst the trace

Superblock

single entrance & multiple exits



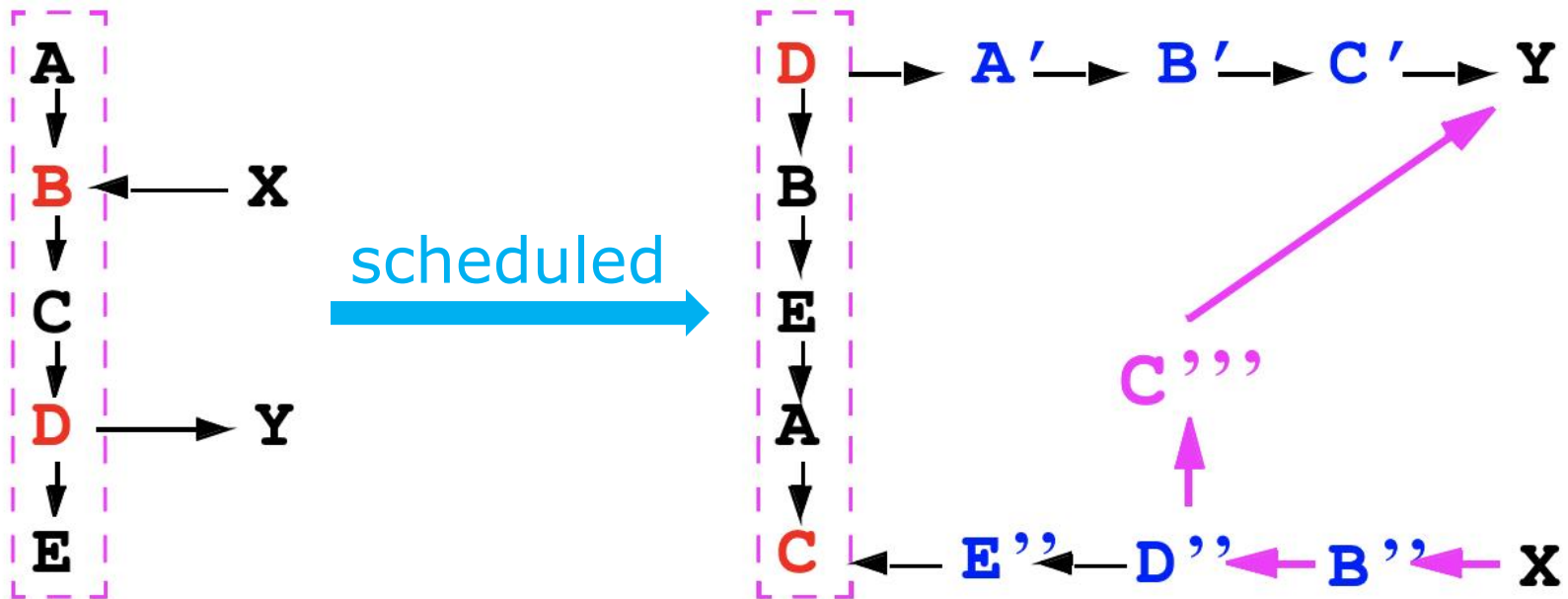
Superblock



single entrance & multiple exits

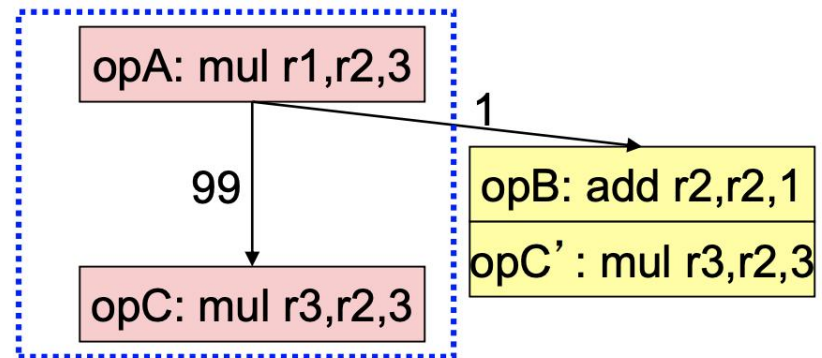
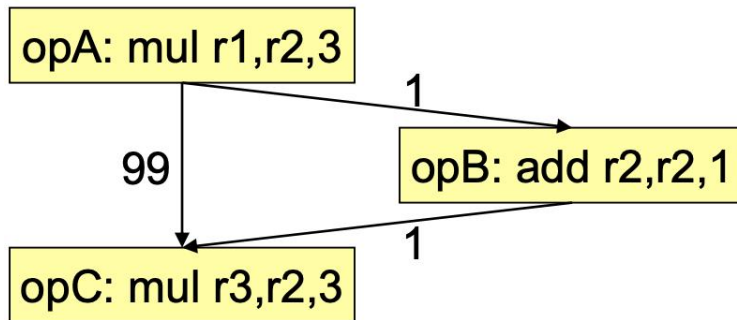
Fixup Code

- copy instructions to ensure correctness of all paths

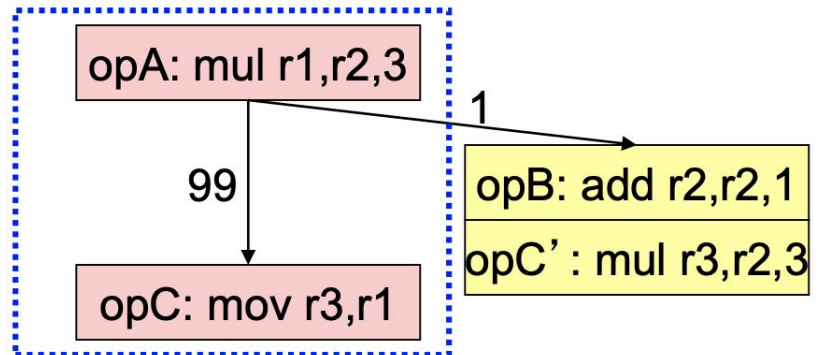


Superblock

- Example with fixup code



- Subexpression elimination



How to handle branches?
when you have to

Control Hazard

- **branches** and jumps
- **Branch hazard**

Instruction	1	2	3	4	5
Load instruction	IF	ID	EX	MEM	WB
Instruction $i + 1$		IF	ID	EX	MEM
Instruction $i + 2$			IF	ID	EX

a branch may or may not change PC to other values other than $PC+4$;

taken branch: changes PC to its target address;

untaken branch: falls through to $PC+4$;

PC is not changed till the end of ID;

Branch Hazard

- **Redo IF**

Branch instruction	IF	ID	EX
Branch successor		IF	IF

essentially a stall



If the branch is untaken,
the stall is unnecessary.

Branch Hazard

- **Redo IF**

Branch instruction	IF	ID	EX
Branch successor		IF	IF

essentially a stall



If the branch is untaken,
the stall is unnecessary.

One stall cycle per branch leads to a 10%~30% perf loss.

Branch Hazard: Solutions

4 simple compile time schemes – 1

- **Freeze or flush the pipeline**

hold or delete any instructions after the branch till the branch dst is known;

Branch instruction	IF	ID	EX
Branch successor			<u>IF</u>

Branch Hazard: Solutions

4 simple compile time schemes – 1

- **Freeze or flush the pipeline**

hold or delete any instructions after the branch till the branch dst is known;

fixed penalty regardless of branch effectiveness;

Branch instruction	IF	ID	EX
Branch successor			<u>IF</u>

Branch Hazard: Solutions

4 simple compile time schemes – 2

- **Predicted-untaken / not-taken**

simply treat every branch as untaken;

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Branch Hazard: Solutions

4 simple compile time schemes – 2

- **Predicted-untaken / not-taken**

simply treat every branch as untaken;

when the branch is untaken,
pipelining as if no hazard.

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Branch Hazard: Solutions

4 simple compile time schemes – 2

- **Predicted-untaken**

but if the branch is taken:

turn fetched instr into a no-op (idle);

restart the IF at the branch target addr

Taken branch instruction	IF	ID	EX	MEM	WB			
Instruction $i + 1$		IF	idle	idle	idle	idle		
Branch target			IF	ID	EX	MEM	WB	
Branch target + 1				IF	ID	EX	MEM	WB
Branch target + 2					IF	ID	EX	MEM
								WB

Branch Hazard: Solutions

4 simple compile time schemes – 3

- **Predicted-taken**

simply treat every branch as taken;

what decides its effectiveness?

Branch Hazard: Solutions

4 simple compile time schemes – 3

- **Predicted-taken**

simply treat every branch as taken;

not apply to the five-stage pipeline
(with branches completed in ID as in 5th
ed);

Branch Hazard: Solutions

4 simple compile time schemes – 3

- **Predicted-taken**

simply treat every branch as taken;

apply to scenarios when branch target
addr is known before branch outcome

(e.g., test branch condition in EX – ALU
as in 6th ed);

Branch Hazard: Solutions

4 simple compile time schemes – 4

- **Delayed branch**

delay the branch execution after the next instruction;

pipelining sequence:

branch instruction

sequential successor

branch target if taken

Branch delay slot
the next instruction
that is independent of
the branch target

executed whether or
not the branch is
taken

Branch Hazard: Solutions

- **Delayed branch**

Untaken branch instruction	IF	ID	EX	MEM	WB
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM
Instruction $i + 2$			IF	ID	EX
Instruction $i + 3$				IF	ID
Instruction $i + 4$					IF
Taken branch instruction	IF	ID	EX	MEM	WB
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM
Branch target			IF	ID	EX
Branch target + 1				IF	ID
Branch target + 2					IF

Branch Hazard: Solutions

- **Example**

20-stage pipeline

1 out of 5 instructions is a branch

5-instruction block ends with a branch

- How long to fetch 500 instructions given various prediction accuracy?

100%

99%

90% https://safari.ethz.ch/digitaltechnik/spring2021/lib/exe/fetch.php?media=onur-digitaldesign_comparch-2021-lecture17b-branch-prediction-i-afterlecture.pdf

Branch Hazard: Solutions

- **Example**

20-stage pipeline

1 out of 5 instructions is a branch

5-instruction block ends with a branch

- How long to fetch 500 instructions given various prediction accuracy?

100%: 1 instr per CC → 500 CC

99%

90% https://safari.ethz.ch/digitaltechnik/spring2021/lib/exe/fetch.php?media=onur-digitaldesign_comparch-2021-lecture17b-branch-prediction-i-afterlecture.pdf

Branch Hazard: Solutions

- **Example**

20-stage pipeline

1 out of 5 instructions is a branch

5-instruction block ends with a branch

- How long to fetch 500 instructions given various prediction accuracy?

100%: 1 instr per CC $\rightarrow$ 500 CC

99%: $500 + 20 \times 1 = 520$ CC

90%

Branch Hazard: Solutions

- **Example**

20-stage pipeline

1 out of 5 instructions is a branch

5-instruction block ends with a branch

- How long to fetch 500 instructions given various prediction accuracy?

100%: 1 instr per CC $\rightarrow$ 500 CC

99%: $500 + 20 \times 1 = 520$ CC

90%: $500 + 20 \times 10 = 700$ CC

Branch Hazard: Solutions

- **Example**

20% branches among all instructions

branch prediction accuracy	branch resolved in 3rd stage	branch resolved in 10th stage
50%		
90%		

Branch Hazard: Solutions

- **Example**

20% branches among all instructions

branch prediction accuracy	branch resolved in 3rd stage	branch resolved in 10th stage
50%	$1 + 0.2 \times 0.5 \times 2$	$1 + 0.2 \times 0.5 \times 9$
90%	$1 + 0.2 \times 0.1 \times 2$	$1 + 0.2 \times 0.1 \times 9$

Branch Hazard: Solutions

- **Example**

20% branches among all instructions

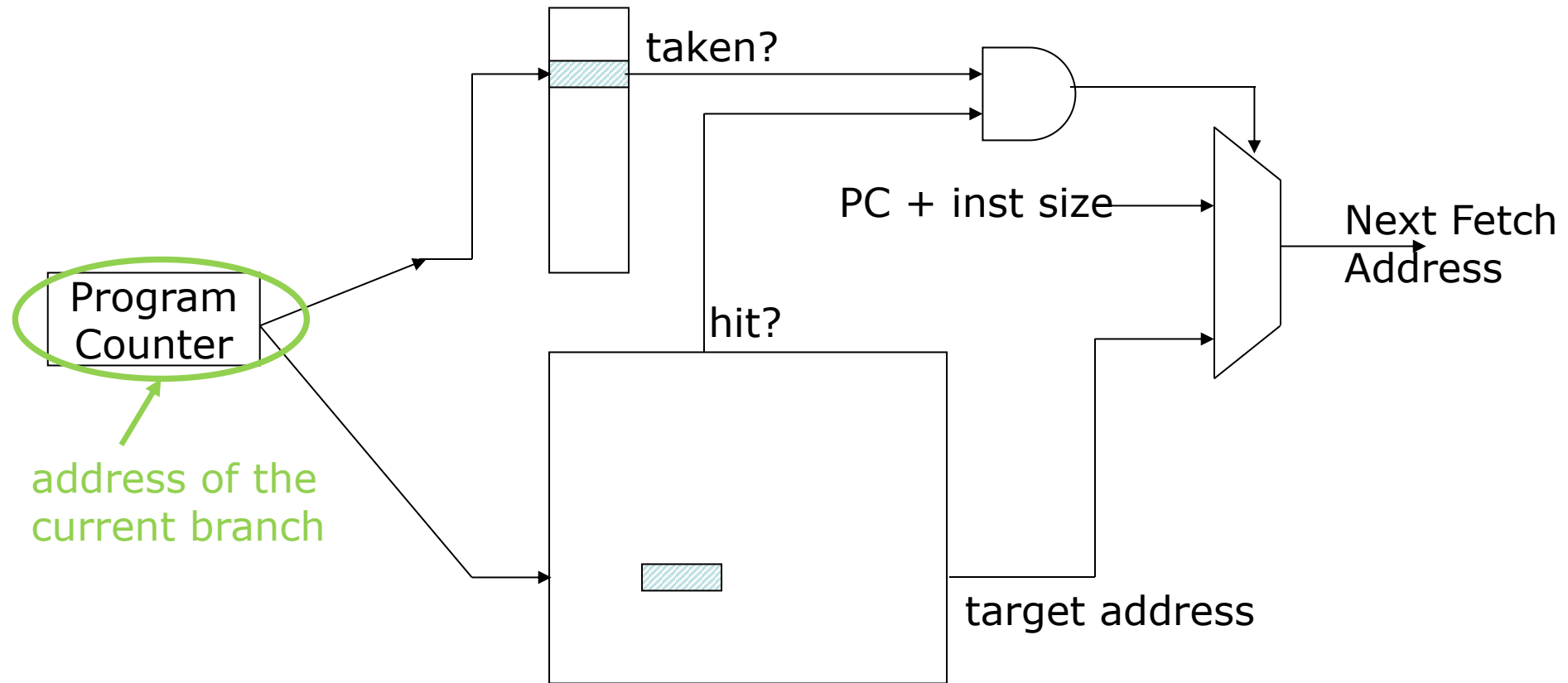
branch prediction accuracy	branch resolved in 3rd stage	branch resolved in 10th stage
50%	1.2	1.9
90%	1.04	1.18

- The deeper branch resolution, the more performance gain with higher branch prediction accuracy

Fetch Stage with BTB & DP

how hardware dynamically helps?

Direction Predictor (taken?)



Cache of Target Addresses (BTB: Branch Target Buffer)

Review

- Integer & floating-point pipelining
- Out-of-order execution
- Hazard & forwarding
- Static scheduling